

Diffusion differentiable resampling

Jennifer Rosina Andersson¹ Zheng Zhao²

Abstract

This paper is concerned with differentiable resampling in the context of sequential Monte Carlo (e.g., particle filtering). Drawing on reparametrisation, we propose a new resampling method that is informative and instantly differentiable, based on a training-free diffusion model surrogate. We theoretically prove that our diffusion resampling method provides a consistent resampling distribution, and we show empirically that it outperforms the state-of-the-art differentiable resampling methods on multiple filtering and parameter estimation benchmarks. Finally, we show that it achieves competitive end-to-end performance when used in learning a complex dynamics-decoder model with high-dimensional image observations.

1. Introduction

Consider a distribution π and a population of weighted samples $\{(w_i, X_i)\}_{i=1}^N \sim \pi$, where $\{X_i\}_{i=1}^N$ are often identically and independently drawn from another proposal distribution, calibrated by the weights. The goal of *resampling* is to transform these weighted samples into an un-weighted set while preserving the original distribution π . In the weak sense, unbiased resampling is defined as a mapping

$$\begin{aligned} \{(w_i, X_i)\}_{i=1}^N &\mapsto \left\{ \left(\frac{1}{N}, X_i^* \right) \right\}_{i=1}^N, \\ \text{s.t. } \mathbb{E} \left[\frac{1}{N} \sum_{i=1}^N \psi(X_i^*) \right] &= \mathbb{E}[\psi(X)] \end{aligned} \quad (1)$$

for any bounded and continuous test function ψ , where $\{X_i^*\}_{i=1}^N$ stands for the re-samples, and $X \sim \pi$.

Resampling is a key component in sequential Monte Carlo (SMC) samplers for state-space models (SSMs, Chopin

& Papaspiliopoulos, 2020). It not only mitigates particle degeneracy in practice, but has also been shown to represent a jump Markov process in a continuous-time limit (Chopin et al., 2022). However, resampling typically hinders (gradient-based) parameter estimation of SSMs due to discrete randomness. For instance, with the commonly used multinomial resampling, one draws indices $I_i \sim \text{Categorical}(w_1, w_2, \dots, w_N)$ independently for $i = 1, 2, \dots, N$, and then defines the resampling mapping by indexing $X_i^* := X_{I_i}$. If the sample X_i^θ (and weight) depend on some unknown parameters θ , the pathwise derivative of the re-sample, $\partial X_i^{\theta,*} / \partial \theta$, is not defined. Moreover, most automatic differentiation libraries (e.g., JAX) will typically drop the undefined derivatives, resulting in erroneous gradient estimates (see, e.g., Naesseth et al., 2018).

Various methods have thus been proposed to make the resampling step differentiable in the context of SMC and SSMs (see recent surveys in Chen & Li, 2025; Brady et al., 2025). One line of work focuses on the expectation derivative $\partial \mathbb{E}[X_i^{\theta,*}] / \partial \theta$, which is often well defined, rather than the pathwise derivative. This can be achieved by combining Fisher’s score (Poyiadjis et al., 2011) and stochastic derivatives (Arya et al., 2022), resulting in, for example, the stop-gradient based method by Ścibior & Wood (2021). However, these REINFORCE-based methods often suffer from high variance and may consequently require a large sample size N .

Another line of work focuses on developing new resampling methods that naturally come with well-defined pathwise derivatives (i.e., reparametrisation). Notable examples include soft (Karkus et al., 2018) and Gumbel-Softmax resampling (Jang et al., 2017). Both methods essentially form an interpolation between multinomial resampling (which has no gradient) and uninformative resampling (which has a gradient) via a calibration parameter. Although they have been empirically shown to work well for certain models, they are fundamentally biased. Crucially, one has to decide on a trade-off between the gradient bias and the statistical performance of the forward resampling mapping. In addition, Zhu et al. (2020) propose to parametrise the resampling with a neural network, which adds both training complexity and additional sources of bias. Kviman et al. (2024) introduce a deterministic resampling, which also comes with irreducible biases (Finke et al., 2026).

¹Department of Information Technology, Uppsala University, Sweden ²Division of Statistics and Machine Learning, Linköping University, Sweden. Correspondence to: Zheng Zhao (趙正) <zheng.zhao@liu.se>.

The perhaps first fully-differentiable-by-construction reparametrisation with consistency guarantees is due to Malik & Pitt (2011), who make a smooth approximation of the empirical cumulative distribution function (CDF) of the weighted samples. However, their method focuses on univariate X . This was later generalised by Li et al. (2024) using kernel jittering to approximate the CDF gradient. In a similar vein, Corenflos et al. (2021) propose an optimal transport (OT) based resampling method. The idea is to learn a (linear) transportation map between the target distribution π and the proposal, and approximate the resampling by an ensemble transformation (Reich, 2013)

$$X_i^* = N \sum_{j=1}^N P_{i,j}^\varepsilon X_j, \quad i = 1, 2, \dots, N, \quad (2)$$

where $P^\varepsilon \in \mathbb{R}^{N \times N}$ denotes the ε -regularised entropic optimal coupling. The main problem of OT-based resampling lies in computation, since one has to compute for the transportation plan P^ε . The cost scales quadratically in the number of samples N with a Sinkhorn implementation, which in turn depends exponentially on the entropy parameter $1/\varepsilon$ (Luo et al., 2023a; Burns & Liang, 2025) to converge. In the context of SMC, the method may not work well when the proposal/reference does not well approximate the target. Li et al. (2024, Fig. 1) also show a (hypothetical) case when linear transformation of OT is insufficient for exploiting the distribution manifold. Nevertheless, this line of work has inspired us to develop a new transportation-based resampling method that avoids these issues.

Other than making the resampling differentiable, one can also modify the SMC algorithm itself to produce smooth estimates of the marginal log likelihood of SSMs. For instance, Klaas et al. (2005) introduce a mixture of SMC proposals to marginalise out the need for resampling, however, in practice, drawing samples from mixture distributions typically also requires discrete sampling. This was addressed by Lai et al. (2022) using implicit reparametrisation, but the approach remains restricted to structured proposals. Overall, this class of methods rely on customised SMC samplers, potentially limiting their applicability in general.

Therefore, our main motivation in this paper is to develop a differentiable resampling method that can be generically applied *as is*, without altering the SMC (or SSMs) construction, sacrificing consistency, or increasing computational cost. We take inspiration from the transportation-based approach (Corenflos et al., 2021), but our key departure here is the transport map construction: it needs not to be *computed* but rather *specified*, thereby mitigating the computation issue. Moreover, our construction allows for integrating additional information of the target into the map to make it statistically more efficient. Our contributions are as follows.

- We introduce *diffusion resampling*, a new reparametrisation paradigm that instantly enables automatic differentiation for $\partial X_i^{\theta,*} / \partial \theta$, and consequently also for the expectation $\partial \mathbb{E}[X_i^{\theta,*}] / \partial \theta$. We apply the method for filtering and gradient-based parameter estimation in state-space models with SMC samplers.
- We prove that our diffusion resampling method is consistent in the number of samples. We show an informative error bound in Wasserstein distance, explicitly quantifying the error propagation of the resampling.
- We empirically validate our method through both ablation and comparison experiments. The results show that our method consistently outperforms the commonly used differentiable resampling methods for both filtering and parameter estimation problems. Notably, our method is computationally efficient and stable, allowing for practical and robust usage in applications.

See Table 20 for a comparison of diffusion resampling to the commonly used differentiable resampling schemes.

2. Diffusion differentiable resampling

In this section we show how we can make use of a diffusion model (without training, cf. Baker et al., 2025; Wan & Zhao, 2025) to construct a differentiable resampling scheme and apply it for sequential Monte Carlo. The idea is akin to Equation (2), which *computes* a linear transportation map, but here we instead use a diffusion model to *construct* a non-linear map. We define the diffusion model via a (forward-time) Langevin stochastic differential equation (SDE)

$$\begin{aligned} dX(t) &= b^2 \nabla \log \pi_{\text{ref}}(X(t)) dt + \sqrt{2} b dW(t), \\ X(0) &\sim \pi \end{aligned} \quad (3)$$

initialised at the target π , where π_{ref} is a user-chosen reference distribution from which we can easily sample (e.g., Gaussian), W is a Brownian motion, and b is a dispersion constant. Under mild conditions (Meyn & Tweedie, 2009), the marginal distribution p_t of $X(t)$ converges to π_{ref} geometrically fast as $t \rightarrow \infty$. Importantly, Song et al. (2021); Anderson (1982) show that we can leverage this construction to sample from π if we can sample p_T at some terminal time $T > 0$ and simulate the reverse-time SDE¹

$$\begin{aligned} dU(t) &= b^2 [-\nabla \log \pi_{\text{ref}}(U(t)) + 2 \nabla \log p_{T-t}(U(t))] dt \\ &\quad + \sqrt{2} b dW(t), \\ U(0) &\sim p_T. \end{aligned} \quad (4)$$

At time T , the marginal distribution q_T of $U(T)$ equals π by construction, since $X(T-t)$ and $U(t)$ solve the same

¹To simplify later analysis we assume that the Brownian motions in Equations (3) and (4) are the same.

Kolmogorov forward equation. Resampling can thus be achieved by sampling from this reversal at T . The challenge is that the score function $\nabla \log p_t$ is intractable, and in the context of generative diffusion models the score is usually learnt from samples of π , introducing demanding computations (Zhao et al., 2025). However, given that we have access to $\{(w_i, X_i)\}_{i=1}^N \sim \pi$, we can approximate the score (Bao et al., 2024) without training according to

$$\begin{aligned} \nabla \log p_t(x_t) &= \frac{\int \nabla \log p_{t|0}(x_t | x_0) p_{t|0}(x_t | x_0) \pi(x_0) dx_0}{\int p_{t|0}(x_t | x_0) \pi(x_0) dx_0} \\ &\approx \frac{\sum_{i=1}^N w_i \nabla \log p_{t|0}(x_t | X_i) p_{t|0}(x_t | X_i)}{\sum_{j=1}^N w_j p_{t|0}(x_t | X_j)}, \end{aligned} \quad (5)$$

where the transition $p_{t|0}$ is analytically tractable for many useful choices of the reference π_{ref} . For later analysis, we define the score approximation by

$$\begin{aligned} \nabla \log p_t(x) &\approx s_N(x, t) \\ &:= \sum_{i=1}^N \alpha_i(x, t) \nabla \log p_{t|0}(x | X_i), \end{aligned} \quad (6)$$

where $\alpha_i(x, t) := w_i p_{t|0}(x | X_i) / \sum_{j=1}^N w_j p_{t|0}(x | X_j)$ stands for the normalised weight. This approximation exactly functions as importance sampling, where $\pi(x_0)$ and $p_{t|0}(\cdot | x_0)$ stand for the prior/proposal and likelihood, respectively. As such, the established $N \rightarrow \infty$ consistency properties of importance sampling apply (Chopin & Papaspiliopoulos, 2020) at least pointwise for $(x, t) \mapsto s_N(x, t)$. Evaluation of the function s has an $O(N)$ computational cost if implemented naively, and a logarithmic cost if implemented in parallel (Lee et al., 2010).

Therefore, the resampling can be approximately achieved by simulating the reverse SDE (4) using the ensemble score s_N in Equation (6) until a terminal time T . Similar to Corenflos et al. (2021), this diffusion process too defines an optimal transportation but in the sense of Jordan–Kinderlehrer–Otto scheme (Jordan et al., 1998). The key distinction is that this map is *given by construction*, and does not need to be *computed* like in OT with Sinkhorn. Although the diffusion also assumes $T \rightarrow \infty$, we show in Section 3 that T scales better than the entropy parameter $1/\varepsilon$ as a function of N .

We summarise the diffusion resampling in Algorithm 1 using a simple Euler–Maruyama discretisation for pedagogy. It is immediate by construction that this function is differentiable, since the only source of randomness is Gaussian which is reparameterisable.

Remark 1. *The ensemble score in Equation (6) characterises a Doob’s h-function:*

$$s_N(x, t) = \nabla \log \sum_{i=1}^N h_i(x, t), \quad (7)$$

Algorithm 1: Diffusion resampling `diffres`

Inputs: Weighted samples $\{(w_i, X_i)\}_{i=1}^N$, reference distribution π_{ref} , time grid $0 = t_0 < t_1 < \dots < t_K = T$, and b .
Outputs: Differentiable re-samples $\{(\frac{1}{N}, X_i^*)\}_{i=1}^N$

```

1 for  $i = 1, 2, \dots, N$  do                                     // parallel
2    $U_{i,0} \sim \pi_{\text{ref}}$ 
3   for  $k = 1, 2, \dots, K$  do
4      $\Delta_k = t_k - t_{k-1}$ 
5     Draw  $\xi_k^i \sim \mathcal{N}(0, 2b^2 \Delta_k I_d)$ 
6      $U_{i,t_k} = U_{i,t_{k-1}} - b^2 [\nabla \log \pi_{\text{ref}}(U_{i,t_{k-1}}) -$ 
7        $2 s_N(U_{i,t_{k-1}}, T - t_{k-1})] \Delta_k + \xi_k^i$ 
8   end
9    $X_i^* \leftarrow U_{i,T}$ 
10 end
```

where $h_i(x, t) := w_i p_{t|0}(x | X_i)$ verifies the martingale (harmonic) property, that is, $\sum_{i=1}^N h_i$ is a valid h -function under SDE (3). As a result, the diffusion resampling process at the terminal time will obtain $\sum_{i=1}^N \gamma_i \delta_{X_i}$ for weights $\{\gamma_i\}_{i=1}^N$ that depend on the spatial location, akin to the OT approach. When setting $p_T(x) = \sum_{i=1}^N h_i(x, T)$, we obtain a special case $\{\gamma_i = w_i : i = 1, \dots, N\}$, and diffusion resampling may thus be viewed as a continuous and differentiable reparametrisation of discrete multinomial resampling. However, the key advantage here is that the diffusion resampling can leverage the additional information from π_{ref} (which implicitly defines a transportation cost and a Rao–Blackwellisation condition) achieving better statistical properties (e.g., variance). See Appendix D for elaboration.

2.1. Differentiable sequential Monte Carlo

The diffusion resampling in Algorithm 1 is particularly useful for SMC and SSMs for two reasons. First, the resampling method is pathwise-differentiable by construction. Moreover, recent advances provide methods for propagating gradients through SDE solvers (see, e.g., Bartosh et al., 2025; Li et al., 2020; Kidger et al., 2021) and these methods have been well implemented in common automatic differentiation libraries (Kidger, 2021). Secondly, we can fully leverage the sequential structure of SMC to adaptively choose for a suitable reference distribution π_{ref} based on any previous SMC marginal distribution. To see these aspects, let us begin by considering a parametrised Feynman–Kac model

$$Q_{0:J}^\theta(z_{0:J}) = \frac{1}{L(\theta)} \prod_{j=0}^J M_j^\theta(z_j | z_{j-1}) G_j^\theta(z_j, z_{j-1}), \quad (8)$$

where M_j^θ and G_j^θ are Markov transition and potential functions, respectively, and $L(\theta)$ is the marginal likelihood that we often aim to maximise. Take an SSM with state tran-

sition $p_\theta(z_j | z_{j-1})$ and measurement $p_\theta(y_j | z_j)$ for example. In this case a bootstrap construction of the corresponding Feynman–Kac model is simply $M_j^\theta(z_j | z_{j-1}) = p_\theta(z_j | z_{j-1})$ and $G_j^\theta(z_j, \cdot) = p_\theta(y_j | z_j)$. One can draw samples of a Feynman–Kac model with an SMC sampler as in Algorithm 2. For detailed exposition of Feynman–Kac models and SMC samplers, we refer the readers to Chopin & Papaspiliopoulos (2020); Del Moral (2004).

Algorithm 2: Differentiable sequential Monte Carlo (SMC) for sampling Feynman–Kac model $Q_{0:J}$.

Inputs: Feynman–Kac model $Q_{0:J}^\theta$, number of samples N , and `diffres`.

Outputs: Weighted samples of $Q_{0:J}^\theta$ and marginal likelihood estimate $L(\theta)$.

```

1 Draw i.i.d. samples  $\{Z_{0,i}\}_{i=1}^N \sim M_0^\theta$ .
2  $L_0(\theta) \leftarrow \sum_{i=1}^N G_0^\theta(Z_{0,i})$ 
3 Weight  $w_{0,i} \leftarrow G_0^\theta(Z_{0,i}) / L_0(\theta)$ 
4 for  $j = 1, 2, \dots, J$  do           //  $i = 1, 2, \dots, N$ 
5     if resampling needed then
6          $Z_{j-1,i}^* \leftarrow \text{diffres}(\{(w_{j-1,i}, Z_{j-1,i})\}_{i=1}^N)$ 
7          $w_{j-1,i} \leftarrow 1 / N$ 
8     else
9          $Z_{j-1,i}^* \leftarrow Z_{j-1,i}$ 
10    end
11    Draw  $Z_{j,i} \sim M_j^\theta(\cdot | Z_{j-1,i}^*)$ 
12     $\bar{w}_{j,i} \leftarrow w_{j-1,i} G_j^\theta(Z_{j,i}, Z_{j-1,i}^*)$ 
13     $L_j(\theta) \leftarrow \sum_{i=1}^N \bar{w}_{j,i}$ 
14    Weight  $w_{j,i} \leftarrow \bar{w}_{j,i} / L_j(\theta)$ 
15 end
16  $L(\theta) \leftarrow \prod_{j=0}^J L_j(\theta)$ 
    
```

This choice of reference π_{ref} is more flexible compared to that of Corenflos et al. (2021) who explicitly use the predictive samples $\{(w_{j-1,i}, Z_{j,i})\}_{i=1}^N$ at the j -th SMC step as the reference to resample $\{(w_{j,i}, Z_{j,i})\}_{i=1}^N$. In contrast, diffusion resampling allows to use, e.g., the posterior samples $\{(w_{j,i}, Z_{j,i})\}_{i=1}^N$ to establish the reference, which can be more informative than the predictive one.

2.2. Mean-reverting Gaussian reference

A remaining question is how to choose the reference distribution π_{ref} . In generative sampling, the reference is usually a unit Normal. However, this becomes suboptimal for resampling when the target distribution π is geometrically far away from $N(0, I_d)$, and as a consequence we would need large enough T for convergence. A more informative choice of π_{ref} is a Gaussian approximation of π . Given that we have access to the target samples $\{(w_i, X_i)\}_{i=1}^N \sim \pi$, we can make use of moment matching, where $\mu_N := \sum_{i=1}^N w_i X_i$

and $\Sigma_N := \sum_{i=1}^N w_i (X_i - \mu_N)(X_i - \mu_N)^\top$ stand for the empirical mean and covariance, respectively (Yang et al., 2013; Kang et al., 2025). We can then choose the reference measure to be

$$\nabla \log \pi_{\text{ref}}(x) = -\Sigma_N^{-1}(x - \mu_N), \quad (9)$$

resulting in a mean-reverting forward SDE

$$dX(t) = -b^2 \Sigma_N^{-1}(X(t) - \mu_N) dt + \sqrt{2}b dW(t) \quad (10)$$

whose forward transition required in Equation (6) is

$$\begin{aligned}
 p_{t|0}(x_t | x_0) &= N(x_t; m_t(x_0), V_t), \\
 m_t(x_0) &:= x_0 e^{-b^2 \Sigma_N^{-1} t} + \mu_N (1 - e^{-b^2 \Sigma_N^{-1} t}), \\
 V_t &:= \Sigma_N (1 - e^{-2b^2 \Sigma_N^{-1} t}).
 \end{aligned}$$

To combine with the SMC in Algorithm 2, we compute μ_N and Σ_N based on $\{(w_{j-1,i}, Z_{j-1,i})\}_{i=1}^N$, see Appendix A for details. It is also possible to leverage any Gaussian filter which is commonly used for approximating the optimal proposal (van der Merwe et al., 2000), to establish the reference. The gist here is to exploit the sequential structure of SMC to adaptively and informatively choose the reference, instead of assuming a static and uninformative one, such as $N(0, I_d)$. Using mean-reverting SDEs for informative generative sampling have also been used in domain applications, such as image restoration (Luo et al., 2023b; 2025).

In practice, to avoid solving the inversion Σ_N^{-1} , we can approximate it as a diagonal, or directly estimate an empirical precision matrix (Yuan & Lin, 2007; Fan et al., 2016). When the Gaussian construction is insufficient for multi-mode targets, one may use a Gaussian mixture, although the associated semigroup needs to be approximated. Another option is to transform with a diffeomorphism to obtain a flexible yet tractable reference process (Deng et al., 2020).

2.3. Exponential integrators

When a Gaussian reference π_{ref} is chosen, the reversal corresponding to the forward Equation (10) will have a semi-linear structure. Hence, we can leverage this structure by applying exponential integrators to accelerate the sampling so as to reduce the computation caused by discretisation. Consider any semi-linear SDE of the form

$$dU(t) = AU(t) + f(U(t), t) dt + \sqrt{2}b dW(t), \quad (11)$$

where in our scenario the linear and non-linear parts respectively correspond to

$$\begin{aligned}
 A &= b^2 \Sigma_N^{-1}, \\
 f(u, t) &= b^2 (2 \nabla \log p_{T-t}(u) - \Sigma_N^{-1} \mu_N).
 \end{aligned} \quad (12)$$

Provided that A is invertible, Jentzen & Kloeden (2009) propose an exponential integrator

$$U_{t_k} = e^{A \Delta_k} U_{t_{k-1}} + A^{-1} (e^{A \Delta_k} - I_d) f(U_{t_{k-1}}) + B_k,$$

$$B_k = \sqrt{2} b \int_{t_{k-1}}^{t_k} e^{(t_k - \tau) A} dW(\tau),$$

where $\Delta_k := t_k - t_{k-1}$, and the Wiener integral B_k simplifies to $B_k \sim N(0, \Sigma_N (e^{2A \Delta_k} - I_d))$. The integrator works effectively if the stiffness of the linear part dominates that of the non-linear part (see conditions in e.g., Buckwar et al., 2011). Indeed, the structure of the approximate score s_N in Equation (6) is essentially a product between a Softmax function and a linear one, which is Lipschitz. However, we note the Lipschitz constant is not uniform for all $t > 0$, resulting in explosive s_N near $t = 0$, such as with V_t . This exponential integrator has been empirically shown to work well for generative diffusion models in practice, for instance by Lu et al. (2025).

In the case when the matrix A is not invertible, which rarely happens for Gaussian π_{ref} but still possibly numerically, one can also use another lower-order integrator by Lord & Rougemont (2004):

$$U_{t_k} = e^{A \Delta_k} U_{t_{k-1}} + \Delta_k e^{A \Delta_k} f(U_{t_{k-1}}, t_{k-1}) + B_k,$$

$$B_k \sim N(0, 2 b^2 e^{2A \Delta_k} \Delta_k),$$

which was also considered by Zhang & Chen (2023).

In light of Remark 1, it is even possible to simulate the resampling SDE fully in continuous time (see, e.g., Schauer et al., 2017; Baker et al., 2024), although most of the currently established techniques are still not (yet) pragmatic enough compared to just using a fine discretisation.

3. Convergence analysis

In this section we analyse the convergence properties of diffusion resampling in Algorithm 1. Recall the ideal resampler in Equation (4)

$$dU(t) = b^2 [-\nabla \log \pi_{\text{ref}}(U(t)) + 2 \nabla \log p_{T-t}(U(t))] dt$$

$$+ \sqrt{2} b dW(t),$$

$$U(0) \sim p_T.$$

and the corresponding approximation

$$d\tilde{U}(t) = b^2 [-\nabla \log \pi_{\text{ref}}(\tilde{U}(t)) + 2 s_N(\tilde{U}(t), T-t)] dt$$

$$+ \sqrt{2} b dW(t), \quad (13)$$

$$\tilde{U}(0) \sim \pi_{\text{ref}}.$$

Here, we have access to the weighted samples $\{(w_i, X_i)\}_{i=1}^N$ from π independent of the considered SDEs. We denote the distributions of $U(t)$ and $\tilde{U}(t)$ by

$q_t = p_{T-t}$ and $\tilde{q}_t$, respectively, and similarly denote the true and approximate re-sample by $U(T)$ and $\tilde{U}(T)$. Clearly, there are two sources of errors: 1) the ensemble score approximation $\nabla \log p_t \approx s_N$, and 2) the initial distribution approximation $p_T \approx \pi_{\text{ref}}$ due to finite time horizon T . For clarity, we here focus on continuous-time analysis, although discretisation errors may be considered within a similar framework (see e.g., Lord et al., 2014). We aim to analyse the geometric distance between the resampling distribution $\tilde{q}_t$ and the target π under these errors in relation to t and N .

Unless otherwise needed, for any parameter $T > 0$ we assume the usual textbook linear growth and Lipschitz conditions on X and U so that a strong solution exists and $\int_0^t |X(\tau)| d\tau$ has finite variance, see, for instance, Karatzas & Shreve (1991, pp. 289) or Øksendal (2007, Thm. 5.2.1). This also ensures a smooth transition density $p_{t|\tau}$ for all $0 \leq \tau < t \leq T$ so that the ensemble score s_N and its gradient in Equation (6) are pointwise well defined. We also assume the existence of the reverse process in the sense of Anderson (1982), i.e., the reversal solves the same Kolmogorov forward equation in reverse time, although the established conditions for this are still implicit (see, e.g., Haussmann & Pardoux, 1986; Millet et al., 1989). Denote the Wasserstein distance by $W_l^i(p, q) = \inf_{\gamma \in \Gamma(p, q)} \mathbb{E}[|X - Y|^l]$ for $(X, Y) \sim \gamma$, where Γ is the set of all couplings of (p, q) . We say a distribution ν is z -strongly log-concave if

$$\langle \nabla \log \nu(x) - \nabla \log \nu(x'), x - x' \rangle \leq -z |x - x'|^2,$$

and we denote it by $\nu \preceq z$. The assumptions that we globally use are as follows.

Assumption 1 (Diffusion conditions). *There exist positive constants $2C_p < C_{\text{ref}} < 2C_{\text{ref}}^- + 2C_p$ such that*

$$|\nabla \log \pi_{\text{ref}}(x) - \nabla \log \pi_{\text{ref}}(x')| \leq C_{\text{ref}} |x - x'|,$$

for all $x, x' \in \mathbb{R}^d$, $\pi_{\text{ref}} \preceq C_{\text{ref}}^-$, and $p_t \preceq C_p$ for all $t \geq 0$.

Assumption 2 (Ensemble score condition). *There exist a constant $r > 0$ and a positive non-increasing smooth function $t \mapsto C_e(t)$, such that*

$$\sup_{x \in \mathbb{R}^d} \mathbb{E}[|\nabla \log p_t(x) - s_N(x, t)|^2]^{1/2} \leq \frac{C_e(t)}{N^r}.$$

for all $t > 0$, where $\mathbb{E}$ takes on the weighted samples.

Recall that the ensemble score defined in Equation (6) is exactly a self-normalised importance sampling, where π is the proposal, $\pi(x_0 | x_t) \propto p_{t|0}(x_t | x_0) \pi(x_0)$ is the target, and $\nabla \log p_{t|0}(x_t | x_0)$ is the test function. The condition in Assumption 2 is thus akin to non-asymptotic variance bounds of importance sampling, and this has been well established by, for example, Agapiou et al. (2017) and Chopin & Papaspiliopoulos (2020, Thm. 8.5). Typically, the Monte

Carlo order is $r = 1/2$. Since the ensemble score may be unbounded as $t \rightarrow 0$, we allow $C_e(t)$ to depend on t without further imposing a specific decay rate. Similar assumptions have been used in De Bortoli (2022, A3) and De Bortoli et al. (2025). We have the following results.

Proposition 1. *Under Assumptions 1 and 2 we have*

$$W_2^2(\tilde{q}_t, q_t) \leq W_2^2(p_T, \pi_{\text{ref}}) e^{b^2 (C_{\text{ref}} - 2C_p) t} + 2b^2 N^{-r} \bar{C}_e(t, T),$$

for all $t \in [0, T)$, where $\bar{C}_e(t, T)$ is defined in Appendix B.

Proof. See Appendix B. $\square$

Proposition 1 quantifies how the two errors due to score approximation and $p_T \approx \pi_{\text{ref}}$ contribute to the resampling error. For a fixed t , the score error diminishes as $N \rightarrow \infty$ at the same rate r but the resampling error has an irreducible bias due to $W_2(p_T, \pi_{\text{ref}})$. Conversely, if we fix N while increasing t , the error bound increases exponentially in t , since the SDE may accumulate the score error over time. This suggests that N should increase fast enough as a function of t to compensate the two errors. A more specific result is thus obtained as follows.

Corollary 1. *Choose $N^{r-c} = \bar{C}_e(t, T)$ for any constant $0 < c < r$, then*

$$W_2^2(\tilde{q}_t, q_t) \leq 2b^2 N^{-c} + e^{b^2 (C_{\text{ref}} - 2C_p) t - 2b^2 C_{\text{ref}} T} W_2^2(\pi, \pi_{\text{ref}}).$$

Furthermore, there exists a linear choice $t \mapsto T(t)$ such that $\lim_{t \rightarrow \infty} W_2(\tilde{q}_t, q_t) = 0$.

Proof. See Appendix C. $\square$

Corollary 1 shows that the diffusion resampling converges as $t \rightarrow \infty$, noting that N is a function of t . The required assumptions are rather mild, and notably, we did not assume any specific decaying rate on C_e . The result also suggests that we should choose the reference π_{ref} close to π , and that the spectral gap of π is not too large. There is likely an optimal b (Lambert function) that minimises the error bound but its explicit value may be hard to know prior to running the algorithm. One may also note that the factor N^{-c} seems suboptimal, as it becomes slower than the importance sampling rate N^{-r} . The factor can be further tightened, as explained in Appendix C.

Remark 2. *When choosing the reference π_{ref} to be a Gaussian, $t \mapsto \bar{C}_e(t, T(t))$ can grow no faster than a polynomial in any of the two arguments, since here $t \mapsto C_e(t)$ decays exponentially. See also De Bortoli et al. (2025, Remark 3).*

Remark 2 show that the required sample size N needs to grow only *polynomially* in T (or equivalently t). This result improves upon that of Corenflos et al. (2021) whose sample size scales *exponentially* in their entropy regularisation term $1/\varepsilon$, and hence potentially implying cheaper computation.

A natural follow-up question is whether we can improve the error bound by eliminating the error due to $p_T \approx \pi_{\text{ref}}$. This may be achieved by using the technique in Corenflos et al. (2025); Zhao et al. (2025), where a forward-backward Gibbs chain $(p_{T|0}, q_{T|0})$ is constructed to avoid explicit sampling from the marginal p_T . This essentially transforms the error $p_T \approx \pi_{\text{ref}}$ into a statistical correlation of the chain. However, since Markov chains typically converge geometrically as well, the required number of chain steps is likely comparable to the diffusion time T (Meyn & Tweedie, 2009; Andrieu et al., 2024). Furthermore, in practice we only have access to the approximate conditional $\tilde{q}_{T|0}$. Nonetheless, we may still benefit computationally from the fact that a fixed T can allow for moderate discretisation.

4. Experiments

In this section we empirically validate our method in both synthetic and real settings. The goal is fourfold. First, we show that the diffusion resampling, regardless of the differentiability, is a useful resampling method and is at least as good as commonly used resampling methods (Section 4.1). Second, we perform ablation experiments to test if the diffusion resampling provides a better estimate of the log marginal likelihood function (Section 4.2). Third and finally, we apply the diffusion resampling for learning neural network-parametrised SSMs using gradient-based optimisation in complex systems (Sections 4.3 and 4.4). Within all these experiments, we focus on comparing to optimal transport (OT, Corenflos et al., 2021), Gumbel-Softmax (Jang et al., 2017), and Soft resampling (Karkus et al., 2018). Our implementations are publicly available at <https://github.com/zgbdm/diffres>.

4.1. Gaussian mixture importance resampling

Consider a target distribution $\pi(x) \propto \phi(x) p(y|x)$ with Gaussian mixture prior $\phi(x) = \sum_{i=1}^c \omega_i N(x; m_i, v_i)$ and likelihood $p(y|x) = N(y; Hx, \Xi)$. The posterior distribution π is also a Gaussian mixture available in closed form (see Appendix G). We use the prior as the proposal to generate importance samples, and then apply the resampling methods to obtain resampled particles. These are compared to samples directly drawn from π , using the sliced 1-Wasserstein distance (SWD) and resampling variance for the mean estimator. We run experiments 100 times independently with sample size $N = 10,000$.

Table 1 shows that the diffusion resampling at $K = 128$

Table 1. Sliced 1-Wasserstein distance (SWD, scaled by 10^{-1}) and resampling variance for the mean estimator (scaled by 10^{-2}), for the Gaussian mixture resampling experiment in Section 4.1.

Method	SWD	Resampling variance
Diffusion ($T = 1, K = 8$)	1.64 ± 0.35	6.87 ± 5.87
Diffusion ($T = 3, K = 128$)	0.80 ± 0.21	3.74 ± 2.99
OT ($\varepsilon = 0.3$)	0.84 ± 0.22	3.42 ± 3.26
OT ($\varepsilon = 0.6$)	0.97 ± 0.21	3.41 ± 3.29
OT ($\varepsilon = 0.9$)	1.14 ± 0.20	3.41 ± 3.29
Multinomial	0.82 ± 0.25	3.78 ± 4.43
Gumbel-Softmax (0.1)	1.40 ± 0.24	3.92 ± 3.74
Soft (0.9)	0.83 ± 0.24	3.75 ± 3.77

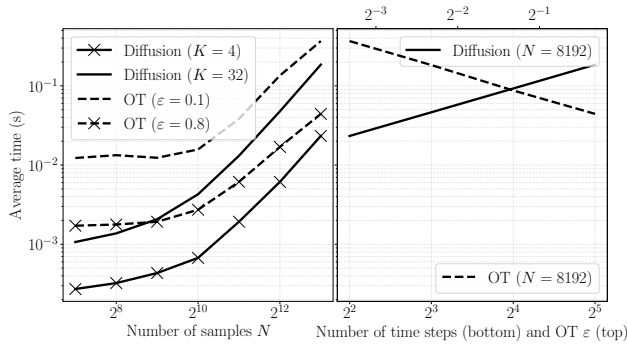


Figure 1. Average running times of diffusion resampling and OT.

gives the best SWD, followed by the baseline multinomial resampling and OT ($\varepsilon = 0.3$). We also observe that the diffusion resampling performance highly depends on the choice of integrator and discretisation, which at $K = 8$ is not superior than OT or multinomial. In terms of resampling variance for estimating the posterior mean, diffusion resampling is not as performant as OT but is still better than the baseline multinomial. OT seems to have stable resampling variance robust in ε . Gumbel-Softmax and the soft resampling methods are not comparable to diffusion or OT. Further results are given in Appendix G.

The time costs of diffusion resampling and OT are shown in Figure 1, and we have two important observations. The left figure shows that both methods roughly scale polynomially in the sample size N , where diffusion ($K = 4$) is better than OT ($\varepsilon = 0.8$). On the right side of the figure, both methods scale linearly in their parameters K and $1/\varepsilon$, which aligns with the summary in Table 20, and the cross point shows that they have a similar computation at $K \approx 6/\varepsilon$ when $N = 8,192$. We also observe a trend that when N increases, the cross point moves left, showing that diffusion scales better than OT in N . Details are given in Appendix H.

Table 2. The errors of loss function, filtering (scaled by 10^{-1}), and parameter estimation (scaled by 10^{-1}) for Section 4.2. Divergent NaN results are explained in Appendix I.

Method	$\ L - \hat{L}\ _2$	Filtering KL	$\ \theta - \hat{\theta}\ _2$
Diffusion ($T = 1, K = 4$)	2.61 ± 2.08	4.94 ± 6.92	1.28 ± 0.70
Diffusion ($T = 2, K = 8$)	2.61 ± 1.89	4.40 ± 4.78	1.29 ± 0.78
Diffusion ($T = 3, K = 8$)	2.55 ± 1.89	4.26 ± 4.49	1.58 ± 0.75
OT ($\varepsilon = 0.4$)	2.64 ± 2.13	5.07 ± 6.21	1.53 ± 1.16
OT ($\varepsilon = 0.8$)	2.68 ± 2.16	5.07 ± 5.70	1.58 ± 1.22
OT ($\varepsilon = 1.6$)	2.75 ± 2.20	5.11 ± 5.17	1.49 ± 0.97
Multinomial	2.80 ± 1.84	5.49 ± 6.87	NaN
Gumbel-Softmax (0.1)	2.79 ± 2.14	4.83 ± 5.76	NaN
Soft (0.9)	2.85 ± 1.80	4.66 ± 5.68	NaN

4.2. Linear Gaussian SSM

We now consider particle filtering and compare different resampling methods on the model

$$\begin{aligned} Z_j | Z_{j-1} &\sim N(z_j; \theta_1 z_{j-1}, I_d), \quad Z_0 \sim N(0, I_d), \\ Y_j | Z_j &\sim N(y_j; \theta_2 z_j, 0.5 I_d) \end{aligned} \quad (14)$$

with parameters $\theta_1 = 0.5$ and $\theta_2 = 1$. For each experiment, we generate a measurement sequence with 128 time steps and apply a Kalman filter to compute the true filtering solution and log-likelihood $L(\theta_1, \theta_2)$ evaluated at Cartesian $\Theta = [\theta_1 - 0.1, \theta_1 + 0.1] \times [\theta_2 - 0.1, \theta_2 + 0.1]$. Three errors are evaluated: 1) the error of log-likelihood function estimate $\|L - \hat{L}\|_2^2 := \int_{\Theta} (L(\theta_1, \theta_2) - \hat{L}(\theta_1, \theta_2))^2 d\theta_1 d\theta_2$, where $\hat{L}$ stands for the estimate by a particle filter with resampling; 2) the KL divergence between the particle samples and the true filtering solution; 3) the estimated parameters $\hat{\theta}$ by L-BFGS compared to the truth under Euclidean norm $\|\theta - \hat{\theta}\|_2$. All results are averaged over 100 independent runs, with 32 particles used. For details see Appendix I.

The results in Table 2 clearly show that the diffusion approach significantly outperforms other resampling methods across all the three metrics with less variance. Importantly, even without considering the differentiability, the diffusion resampling already gives the best filtering estimate, showing that it is a useful resampling method in itself. This is likely due to the fact that in diffusion resampling, the reference is given by the current posterior samples instead of the predictive samples like OT. In contrast to the Gaussian mixture experiment where the diffusion needs larger K to be comparable to OT, the needs for fine discretisation is moderate for this model. Also based on Figure 1, the diffusion resampling achieves substantially better results than OT when evaluated at the same level of computational cost.

We also observe that almost all the resampling methods encounter divergent parameter estimations to some extent (see more statistics in Appendix I). In particular, the divergence of Soft and Gumbel-Softmax is too significant to give a meaningful result, and these entries are thus marked as NaN. The reason is due to the quasi-Newton optimiser

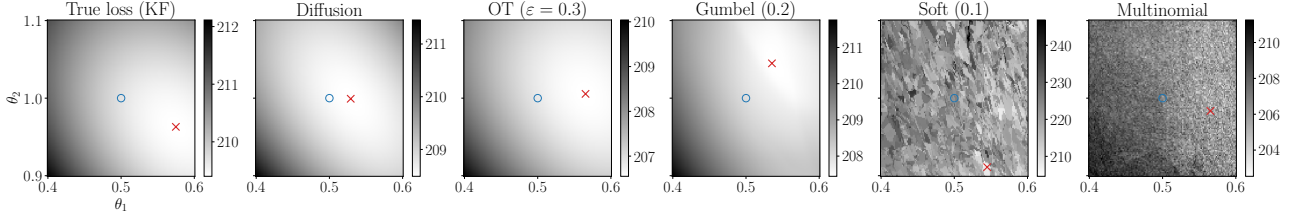


Figure 2. Visualisation of the particle filter estimated log-likelihoods using different resampling methods associated with the LGSSM experiment. Blue circle $\circ$ stands for the true parameter, while the red cross $\times$ represents the minimum of the estimated loss function. We see in this example that the diffusion resampling gives an estimate closest to both the true parameter and the true loss minimum.

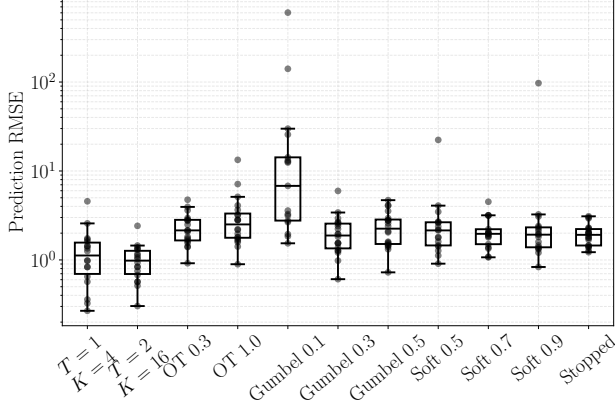


Figure 3. Root mean square errors of the learnt prey-predator models over independent runs (scatter points).

L-BFGS-B which heavily depends on stable and accurate gradient estimates (Xie et al., 2020). This shows that the diffusion and OT approaches are more applicable for generic optimisers, such as second-order ones. Therefore, in the next sections we will focus on comparison using first-order gradient-based methods.

4.3. Prey-predator model

Consider a more challenging Lotka–Volterra model

$$\begin{aligned} dC(t) &= C(t) (\alpha - \beta R(t)) dt + \sigma C(t) dW_1(t), \\ dR(t) &= R(t) (\zeta C(t) - \gamma) dt + \sigma R(t) dW_2(t), \\ Y_j &\sim \text{Poisson}(\lambda(C(t_j), R(t_j))), \end{aligned} \quad (15)$$

where the configuration is detailed in Appendix J. We generate data at time $t \in [0, 3]$ discretised by Milstein’s method with 256 steps. We model the dynamics entirely by a neural network, without assuming known the dynamic structure. After the neural network has been learnt, we make 100 predictions from the learnt model and then compare them to a reference trajectory generated by the true dynamics. All results are averaged over 20 independent runs, with $N = 64$ particles used. We have also compared to the REINFORCE framework implemented with a stopped gradient method by Ścibior & Wood (2021).

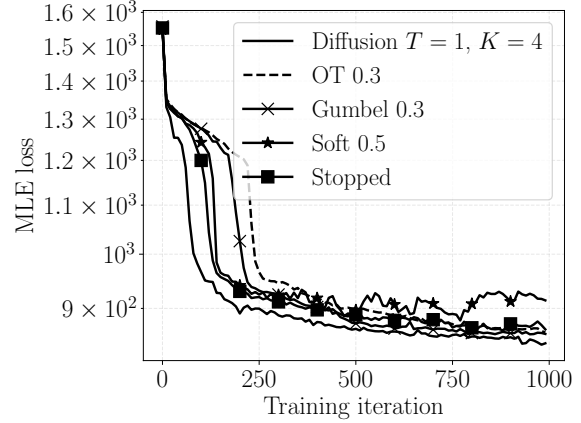


Figure 4. Loss traces (median over all runs) for training the prey-predator model. The training enabled by the diffusion resampling achieves the lowest and stablest.

Results are summarised in Figures 3 and 4. The first figure shows that the trained model using the diffusion resampling achieves the lowest prediction error and is significantly more stable than the baselines. The second figure aligns with the results from the first figure, showing that the training loss with diffusion resampling consistently lower bounds that of the other methods, and is more stable.

4.4. Vision-based pendulum dynamics tracking

Next, we consider the problem of learning the dynamics of a physical system from high-dimensional and structural image observations. We simulate pendulum dynamics described by a discrete-time SSM

$$\begin{aligned} Z_j^{(1)} &= Z_{j-1}^{(1)} + Z_{j-1}^{(2)} \Delta_j + \zeta_j^{(1)}, \\ Z_j^{(2)} &= Z_{j-1}^{(2)} - \frac{g}{l} \sin(Z_{j-1}^{(1)}) \Delta_j + \zeta_j^{(2)}, \\ Y_j | Z_j &\sim \mathcal{N}(r(Z_j), \sigma_{\text{obs}}^2 I_{32 \times 32}), \end{aligned} \quad (16)$$

where $Z_j := \begin{bmatrix} Z_j^{(1)} & Z_j^{(2)} \end{bmatrix}$ represents the state defined by the angle and angular velocity, Δ_j is the discretisation step, and $\zeta_j \sim \mathcal{N}(0, \Delta_j \Lambda_\zeta)$ is the process noise. The true observation function r generates 32×32 snapshots of the pendulum. We generate a trajectory of observations $Y_{0:j}$

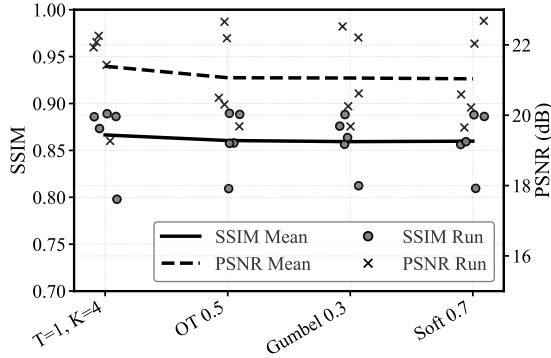


Figure 5. Mean prediction SSIM and PSNR for the best (by mean) model configuration of each resampler in the first experiment setting. Individual runs are shown as scatter points.

over $J = 256$ steps. In contrast to Section 4.3, we learn both the underlying dynamics and the observation model, and we parametrise both the state transition, $Z_j = f_\theta(Z_{j-1}, \zeta_j)$, and the decoder, $r \approx r_\phi$, by neural networks. Parameters θ and ϕ are learnt by minimising the negative log-likelihood estimated by the particle filter with resampling. For completeness, we run experiments in two regimes: the first uses adaptive resampling under a tempered observation likelihood, and the second is a more challenging setting with resampling at nearly every filtering step. The learnt models are evaluated using the average structural similarity index (SSIM) and peak signal-to-noise ratio (PSNR) on their predicted image sequences.

Our results demonstrate that diffusion resampling integrates effectively into this complex, high-dimensional SMC learning pipeline, enabling stable optimisation and achieving competitive performance relative to state-of-the-art differentiable resampling baselines. Figure 6 shows an accurate and high-fidelity mean image sequence reconstructed from a latent pendulum dynamics model and decoder learnt with diffusion resampling embedded in the optimisation process. The comparative results in Figure 5 show that, among the strongest configurations of each resampling class, diffusion resampling achieves mean SSIM and PSNR at least comparable to the baselines. See Appendix K for further details and results. We also include additional large-scale experiments in Appendix L and Appendix M.

5. Related work

In concurrent work, Gourevitch et al. (2026) propose a differentiable reparameterisation of categorical distributions using stochastic interpolants. While our diffusion resampler similarly constitutes a differentiable relaxation of categorical sampling, the key departure is that their closed-form denoiser is derived under one-hot encoded samples $\{X_i\}_{i=1}^N$, whereas we focus on samples in $\mathbb{R}^d$. We further consider

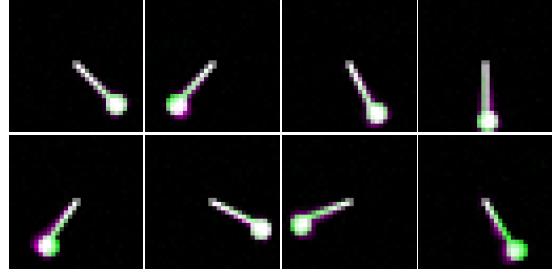


Figure 6. Qualitative comparison of the learnt pendulum dynamics. The ground truth (green) is overlaid with model predictions (purple). White pixels indicate perfect alignment, while coloured regions highlight positional discrepancies (e.g., phase lag). Snapshots are shown at eight evenly spaced time points over the 4 second simulation (read from left to right and top to bottom).

the case when the empirical measure approximates an underlying continuous distribution as $N \rightarrow \infty$, while Gourevitch et al. (2026) mostly focus on discrete categorical distributions. There exists no exact reparametrisation producing the true expectation gradient *under the categorical measure*, however, a consistent reparametrisation (e.g., ours) exists converging to the true expectation gradient *under the underlying continuous measure*.

Another work closely related to ours is that by Wan & Zhao (2025), who also leverages a diffusion model for differentiable resampling within the SMC framework. While empirically powerful, their approach relies on a *trained* conditional diffusion model, which introduces bias, breaks consistency guarantees, and adds substantial computational cost. In addition, their method introduces a further challenge in that the resampling gradient should also be propagated through the diffusion training. In contrast, our method focuses on a statistically grounded diffusion resampling scheme without training, and is more informative.

6. Conclusion

In this paper we have proposed a new differentiable-by-construction, computationally efficient, and informative resampling method built on diffusion models. We have explicitly quantified a convergent error bound in the sample size and diffusion parameters. Our experiments verify that the proposed method outperforms the state-of-the-art differentiable resampling methods for filtering and parameter estimation of state-space models.

Limitations and future work. We observe that propagating gradients through the diffusion resampling can be numerically sensitive to the SDE solver. This could potentially be mitigated by, e.g., reversible adjoint Heun’s method (Kidger et al., 2021) or related variants. We also note that the ensemble score may not be the only choice to achieve diffusion resampling.

Acknowledgements

This work was partially supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg (KAW) Foundation. Computations were enabled by the supercomputing resource Berzelius provided by National Supercomputer Centre at Linköping University and the KAW foundation. We also thank the Stiftelsen G.S Magnusons fond (MG2024-0035).

Authors' contributions are as follows. JA verified the theoretical results, wrote Related work, and conducted the pendulum and weather forecasting experiments. ZZ came up with the idea and developed the method, wrote the initial draft, proved the theoretical results, and performed the other experiments. All authors contributed to and revised the final manuscript.

Impact statement

The work is concerned with a fundamental problem within statistics and machine learning. It does not directly lead to any ethical and societal concerns needed to be explicitly discussed here.

References

- Agapiou, S., Papaspiliopoulos, O., Sanz-Alonso, D., and Stuart, A. M. Importance sampling: Intrinsic dimension and computational cost. *Statistical Science*, 32(3):405–431, 2017.
- Ambrosio, L., Gigli, N., and Savaré, G. *Gradient flows in metric spaces and in the space of probability measures*. Lectures in Mathematics ETH Zürich. Birkhäuser Verlag, 2nd edition, 2008.
- Anderson, B. D. O. Reverse-time diffusion equation models. *Stochastic Processes and their Applications*, 12(3):313–326, 1982.
- Andrieu, C., Lee, A., Power, S., and Wang, A. Q. Explicit convergence bounds for Metropolis Markov chains: Isoperimetry, spectral gaps and profiles. *The Annals of Applied Probability*, 34(4):4022–4071, 2024.
- Arya, G., Schauer, M., Schäfer, F., and Rackauckas, C. Automatic differentiation of programs with discrete randomness. In *Advances in Neural Information Processing Systems*, volume 35, pp. 10435–10447. Curran Associates, Inc., 2022.
- Baker, E. L., Yang, G., Severinsen, M. L., Hipsley, C. A., and Sommer, S. Conditioning non-linear and infinite-dimensional diffusion processes. In *Advances in Neural Information Processing Systems*, volume 37, pp. 10801–10826. Curran Associates, Inc., 2024.
- Baker, E. L., Schauer, M., and Sommer, S. Score matching for bridges without learning time-reversals. In *Proceedings of the 28th International Conference on Artificial Intelligence and Statistics*, volume 258, pp. 775–783. PMLR, 03–05 May 2025.
- Bao, F., Zhang, Z., and Zhang, G. An ensemble score filter for tracking high-dimensional nonlinear dynamical systems. *Computer Methods in Applied Mechanics and Engineering*, 432:117447, 2024.
- Bartosh, G., Vetrov, D., and Naesseth, C. A. SDE matching: Scalable and simulation-free training of latent stochastic differential equations. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267, pp. 3054–3070. PMLR, 13–19 Jul 2025.
- Blondel, M., Berthet, Q., Cuturi, M., Frostig, R., Hoyer, S., Llinares-López, F., Pedregosa, F., and Vert, J.-P. Efficient and modular implicit differentiation. *arXiv preprint arXiv:2105.15183*, 2021.
- Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., and Zhang, Q. JAX: composable transformations of Python+NumPy programs, 2018. URL <http://github.com/jax-ml/jax>.
- Brady, J.-J., Cox, B., Elvira, V., and Li, Y. PyDPF: A Python package for differentiable particle filtering. *arXiv preprint arXiv:2510.25693*, 2025.
- Buckwar, E., Riedler, M. G., and Kloeden, P. E. The numerical stability of stochastic ordinary differential equations with additive noise. *Stochastics and Dynamics*, 11 (02n03):265–281, 2011.
- Burns, M. X. and Liang, J. Linear-space extragradient methods for fast, large-scale optimal transport. *arXiv preprint arXiv:2511.11359*, 2025.
- Chang, P. G., Murphy, K. P., and Jones, M. On diagonal approximations to the extended Kalman filter for online training of Bayesian neural networks. In *Continual Lifelong Learning Workshop at ACML 2022*, 2022.
- Chen, S., Chewi, S., Li, J., Li, Y., Salim, A., and Zhang, A. Sampling is as easy as learning the score: theory for diffusion models with minimal data assumptions. In *Proceedings of the 11th International Conference on Learning Representations*, 2023.
- Chen, X. and Li, Y. An overview of differentiable particle filters for data-adaptive sequential Bayesian inference. *Foundations of Data Science*, 7(4):915–943, 2025.
- Chopin, N. and Papaspiliopoulos, O. *An introduction to sequential Monte Carlo*. Springer Series in Statistics. Springer, 2020.

- Chopin, N., Singh, S. S., Soto, T., and Vihola, M. On resampling schemes for particle filters with weakly informative observations. *The Annals of Statistics*, 50(6):3197–3222, 2022.
- Corenflos, A., Thornton, J., Deligiannidis, G., and Doucet, A. Differentiable particle filtering via entropy-regularized optimal transport. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139, pp. 2100–2111. PMLR, 18–24 Jul 2021.
- Corenflos, A., Zhao, Z., Särkkä, S., Sjölund, J., and Schön, T. B. Conditioning diffusion models by explicit forward-backward bridging. In *Proceedings of the 28th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 258, pp. 3709–3717. PMLR, 2025.
- Course, K. and Nair, P. B. Amortized reparameterization: Efficient and scalable variational inference for latent SDEs. In *Advances in Neural Information Processing Systems*, volume 36, pp. 78296–78318. Curran Associates, Inc., 2023.
- Cuturi, M., Meng-Papaxanthos, L., Tian, Y., Bunne, C., Davis, G., and Teboul, O. Optimal transport tools (OTT): A JAX toolbox for all things Wasserstein. *arXiv preprint arXiv:2201.12324*, 2022.
- De Bortoli, V. Convergence of denoising diffusion models under the manifold hypothesis. *Transactions on Machine Learning Research*, 2022.
- De Bortoli, V., Elie, R., Kazeykina, A., Ren, Z., and Zhang, J. Dimension-free error estimate for diffusion model and optimal scheduling. *arXiv preprint arXiv:2512.01820*, 2025.
- Del Moral, P. *Feynman-Kac formulae: genealogical and interacting particle systems with applications*. Springer New York, 2004.
- Deng, R., Chang, B., Brubaker, M. A., Mori, G., and Lehmman, A. Modeling continuous stochastic processes with dynamic normalizing flows. In *Advances in Neural Information Processing Systems*, volume 33, pp. 7805–7815. Curran Associates, Inc., 2020.
- Dragomir, S. S. *Some Grönwall type inequalities and applications*. Nova Science Publisher, 2003.
- Fan, J., Liao, Y., and Liu, H. An overview of the estimation of large covariance and precision matrices. *The Econometrics Journal*, 19(1):C1–C32, 2016.
- Finke, A., Kviman, O., Branchini, N., and Elvira, V. On the bias of variational resampling. In *Proceedings of The 29th International Conference on Artificial Intelligence and Statistics*. PMLR, 2026.
- Freitas, J. F. G. d., Niranjan, M., Gee, A. H., and Doucet, A. Sequential Monte Carlo methods to train neural network models. *Neural Computation*, 12(4):955–993, 2000.
- Gourevitch, S., Durmus, A., Moulines, E., Olsson, J., and Janati, Y. Categorical reparameterization with denoising diffusion models. *arXiv preprint arxiv:2601.00781*, 2026.
- Greydanus, S., Dzamba, M., and Yosinski, J. Hamiltonian neural networks. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- Hausmann, U. G. and Pardoux, E. Time reversal of diffusions. *The Annals of Probability*, 14(4):1188–1205, 1986.
- Heek, J., Levskaya, A., Oliver, A., Ritter, M., Rondepierre, B., Steiner, A., and van Zee, M. Flax: A neural network library and ecosystem for JAX, 2024. URL <http://github.com/google/flax>.
- Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, volume 33, pp. 6840–6851. Curran Associates, Inc., 2020.
- Jang, E., Gu, S., and Poole, B. Categorical reparameterization with Gumbel-Softmax. In *Proceedings of the 5th International Conference on Learning Representations*, 2017.
- Jentzen, A. and Kloeden, P. E. Overcoming the order barrier in the numerical approximation of stochastic partial differential equations with additive space-time noise. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 465(2102):649–667, 2009.
- Jordan, R., Kinderlehrer, D., and Otto, F. The variational formulation of the Fokker–Planck equation. *SIAM Journal on Mathematical Analysis*, 29(1):1–17, 1998.
- Kang, J., Jiao, X., and Yau, S. S.-T. Estimation of the linear system via optimal transportation and its application for missing data observations. *IEEE Transactions on Automatic Control*, 70(9):5644–5659, 2025.
- Karatzas, I. and Shreve, S. E. *Brownian motion and stochastic calculus*, volume 113 of *Graduate Texts in Mathematics*. Springer-Verlag New York, 2nd edition, 1991.
- Karkus, P., Hsu, D., and Lee, W. S. Particle filter networks with application to visual localization. In *Proceedings of The 2nd Conference on Robot Learning*, volume 87, pp. 169–178. PMLR, 29–31 Oct 2018.
- Kidger, P. *On neural differential equations*. PhD thesis, University of Oxford, 2021.

- Kidger, P., Foster, J., Li, X. C., and Lyons, T. Efficient and accurate gradients for neural SDEs. In *Advances in Neural Information Processing Systems*, volume 34, pp. 18747–18761. Curran Associates, Inc., 2021.
- Klaas, M., Freitas, N. d., and Doucet, A. Toward practical n^2 Monte Carlo: the marginal particle filter. In *Proceedings of the 21st Conference on Uncertainty in Artificial Intelligence*, pp. 308–315, 2005.
- Kviman, O., Branchini, N., Elvira, V., and Lagergren, J. Variational resampling. In *Proceedings of The 27th International Conference on Artificial Intelligence and Statistics*, volume 238, pp. 3286–3294. PMLR, 02–04 May 2024.
- Lai, J., Domke, J., and Sheldon, D. Variational marginal particle filters. In *Proceedings of The 25th International Conference on Artificial Intelligence and Statistics*, volume 151 of *Proceedings of Machine Learning Research*, pp. 875–895. PMLR, 2022.
- Lee, A., Yau, C., Giles, M. B., Doucet, A., and Holmes, C. C. On the utility of graphics cards to perform massively parallel simulation of advanced Monte Carlo methods. *Journal of Computational and Graphical Statistics*, 19(4):769–789, 2010.
- Li, X., Wong, T.-K. L., Chen, R. T. Q., and Duvenaud, D. Scalable gradients for stochastic differential equations. In *Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics*, volume 108, pp. 3870–3882. PMLR, 26–28 Aug 2020.
- Li, Y., Wang, W., Deng, K., and Liu, J. S. Differentiable particle filters with smoothly jittered resampling. *Statistica Sinica*, 34:1241–1262, 2024.
- Liu, Q. Stein variational gradient descent as gradient flow. In *Advances in Neural Information Processing Systems*, volume 30, pp. 3118–3126. Curran Associates, Inc., 2017.
- Lord, G. J. and Rougemont, J. A numerical scheme for stochastic PDEs with Gevrey regularity. *IMA Journal of Numerical Analysis*, 24(4):587–604, 2004.
- Lord, G. J., Powell, C. E., and Shardlow, T. *An introduction to computational stochastic PDEs*, volume 50 of *Cambridge Texts in Applied Mathematics*. Cambridge University Press, 2014.
- Lu, C., Zhou, Y., Bao, F., Chen, J., Li, C., and Zhu, J. DPM-solver++: Fast solver for guided sampling of diffusion probabilistic models. *Machine Intelligence Research*, 22: 730–751, 2025.
- Luo, Y., Xie, Y., and Huo, X. Improved rate of first order algorithms for entropic optimal transport. In *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics*, volume 206, pp. 2723–2750. PMLR, 25–27 Apr 2023a.
- Luo, Z., Gustafsson, F. K., Zhao, Z., Sjölund, J., and Schön, T. B. Image restoration with mean-reverting stochastic differential equations. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pp. 23045–23066. PMLR, 2023b.
- Luo, Z., Gustafsson, F. K., Zhao, Z., Sjölund, J., and Schön, T. B. Taming diffusion models for image restoration: a review. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 383(2299):20240358, 2025.
- Malik, S. and Pitt, M. K. Particle filters for continuous likelihood evaluation and maximisation. *Journal of Econometrics*, 165(2):190–209, 2011.
- Meyn, S. and Tweedie, R. L. *Markov chain and stochastic stability*. Cambridge University Press, 2nd edition, 2009.
- Millet, A., Nualart, D., and Sanz, M. Integration by parts and time reversal for diffusion processes. *The Annals of Probability*, 17(1):208–238, 1989.
- Naesseth, C., Linderman, S., Ranganath, R., and Blei, D. Variational sequential Monte Carlo. In *Proceedings of the 21st International Conference on Artificial Intelligence and Statistics*, volume 84, pp. 968–977. PMLR, 09–11 Apr 2018.
- Ng, K., van der Heide, C., Hodgkinson, L., and Wei, S. Temperature optimization for Bayesian deep learning. In *Proceedings of the 41st Conference on Uncertainty in Artificial Intelligence*, volume 286, pp. 3155–3181. PMLR, 21–25 Jul 2025.
- Øksendal, B. *Stochastic differential equations: an introduction with applications*. Universitext. Springer-Verlag Berlin Heidelberg, 6th edition, 2007.
- Poyiadjis, G., Doucet, A., and Singh, S. S. Particle approximations of the score and observed information matrix in state space models with application to parameter estimation. *Biometrika*, 98(1):65–80, 2011.
- Rasp, S., Dueben, P. D., Scher, S., Weyn, J. A., Mouatadid, S., and Thuerey, N. Weatherbench: A benchmark data set for data-driven weather forecasting. *Journal of Advances in Modeling Earth Systems*, 2020.
- Reich, S. A nonparametric ensemble transform method for Bayesian inference. *SIAM Journal on Scientific Computing*, 35(4):A2013–A2024, 2013.

- Rosato, C., Devlin, L., Beraud, V., Horridge, P., Schön, T. B., and Maskell, S. Efficient learning of the parameters of non-linear models using differentiable resampling in particle filters. *IEEE Transactions on Signal Processing*, 70:3676–3692, 2022.
- Schauer, M., van der Meulen, F., and van Zanten, H. Guided proposals for simulating multi-dimensional diffusion bridges. *Bernoulli*, 23(4A):2917–2950, 2017.
- Ścibior, A. and Wood, F. Differentiable particle filtering without modifying the forward pass. *arXiv preprint arXiv:2106.10314*, 2021.
- Sharma, M., Farquhar, S., Nalisnick, E., and Rainforth, T. Do Bayesian neural networks need to be fully stochastic? In *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics*, volume 206, pp. 7694–7722. PMLR, 25–27 Apr 2023.
- Sitzmann, V., Martel, J., Bergman, A., Lindell, D., and Wetzstein, G. Implicit neural representations with periodic activation functions. In *Advances in Neural Information Processing Systems*, volume 33, pp. 7462–7473. Curran Associates, Inc., 2020.
- Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., and Poole, B. Score-based generative modeling through stochastic differential equations. In *Proceedings of the 9th International Conference on Learning Representations*, 2021.
- van der Merwe, R., Doucet, A., de Freitas, N., and Wan, E. The unscented particle filter. In *Advances in Neural Information Processing Systems*, volume 13. MIT Press, 2000.
- von Renesse, M.-K. and Sturm, K.-T. Transport inequalities, gradient estimates, entropy and Ricci curvature. *Communications on Pure and Applied Mathematics*, 58(7): 923–940, 2005.
- Wan, Z. and Zhao, L. DiffPF: Differentiable particle filtering with generative sampling via conditional diffusion models. *arXiv preprint arXiv:2507.15716*, 2025.
- Xie, Y., Byrd, R. H., and Nocedal, J. Analysis of the BFGS method with errors. *SIAM Journal on Optimization*, 30(1):182–209, 2020.
- Yang, T., Mehta, P. G., and Meyn, S. P. Feedback particle filter. *IEEE Transactions on Automatic Control*, 58(10): 2465–2480, 2013.
- Yuan, M. and Lin, Y. Model selection and estimation in the Gaussian graphical model. *Biometrika*, 94(1):19–35, 2007.
- Zhang, Q. and Chen, Y. Fast sampling of diffusion models with exponential integrator. In *Proceedings of the 11th International Conference on Learning Representations*, 2023.
- Zhao, Z. Generative diffusion posterior sampling for informative likelihoods. *Communications in Information and Systems*, 2025. Special issue for celebrating Thomas Kailath’s 90th birthday. In press.
- Zhao, Z. and Sarmarvuori, J. Stochastic filtering with moment representation. *arXiv preprint arXiv:2303.13895*, 2023.
- Zhao, Z., Mair, S., Schön, T. B., and Sjölund, J. On Feynman–Kac training of partial Bayesian neural networks. In *Proceedings of the 27th International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 238, pp. 3223–3231. PMLR, 2024.
- Zhao, Z., Luo, Z., Sjölund, J., and Schön, T. B. Conditional sampling within generative diffusion models. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 383(2299):20240329, 2025.
- Zhu, M., Murphy, K., and Jonschkowski, R. Towards differentiable resampling. *arXiv preprint arXiv:2004.11938*, 2020.

A. Diffusion resampling with Gaussian reference

For easy reproducibility, we present the diffusion resampling specifically with the mean-reverting Gaussian reference π_{ref} in the following algorithm. In practice, the algorithm is always implemented for log weights.

Algorithm 3: Diffusion resampling `diffres` with Gaussian reference

Inputs: Weighted samples $\{(w_i, X_i)\}_{i=1}^n$ and time grid $0 = t_0 < t_1 < \dots < t_K = T$.

Outputs: Differentiable re-samples $\{(\frac{1}{N}, X_i^*)\}_{i=1}^n$

```

1  $\mu_N \leftarrow \sum_{i=1}^N w_i X_i$ 
2  $\Sigma_N \leftarrow \sum_{i=1}^N w_i (X_i - \mu_N) (X_i - \mu_N)^\top$  // Or other Gaussian approximation methods
3 def  $s(x, t)$ :
4      $\alpha_i(x, t) \leftarrow w_i \mathcal{N}(x; m_t(X_i), V_t)$  // parallel for  $i = 1, 2, \dots, N$ 
5      $\alpha_i(x, t) \leftarrow \alpha_i(x, t) / \sum_{j=1}^N \alpha_j(x, t)$ 
6     return  $-\sum_{i=1}^N \alpha_i(x, t) V_t^{-1} (x - m_t(X_i))$ 
7 for  $i = 1, 2, \dots, N$  do // parallel for  $i = 1, 2, \dots, N$ 
8      $U_{i,0} \sim \mathcal{N}(\mu_N, \Sigma_N)$ 
9     for  $k = 1, 2, \dots, K$  do
10         $\Delta_k = t_k - t_{k-1}$ 
11        Draw  $\xi_k^i \sim \mathcal{N}(0, 2b^2 \Delta_k I_d)$ 
12         $U_{i,t_k} = U_{i,t_{k-1}} + b^2 [\Sigma_N^{-1} (U_{i,t_{k-1}} - \mu_N) + 2s(U_{i,t_{k-1}}, T - t_{k-1})] \Delta_k + \xi_k^i$ 
13        // Or any other SDE solver for Equation (4)
14    end
15     $X_i^* \leftarrow U_{i,T}$ 
16 end
    
```

B. Proof of Proposition 1

For any $t \in [0, T)$, define a residual $h_t := U(t) - \tilde{U}(t)$, and recall that they both are driven by the same Brownian motion. We have

$$|h_t| \leq |U(0) - \tilde{U}(0)| + b^2 \left(\int_0^t |\nabla \log \pi_{\text{ref}}(\tilde{U}(\tau)) - \nabla \log \pi_{\text{ref}}(U(\tau))| d\tau + 2 \int_0^t |\nabla \log p_{T-\tau}(U(\tau)) - s_N(\tilde{U}(\tau), T - \tau)| d\tau \right) \quad (17)$$

By Itô's formula we have

$$|h_t|^2 = |h_0|^2 + 2 \int_0^t \langle h_\tau, h'_\tau \rangle d\tau, \quad (18)$$

where h'_τ stands for the drift of h_τ . We obtain

$$\begin{aligned} |h_t|^2 &= |h_0|^2 + 2b^2 \int_0^t \langle h_\tau, \nabla \log \pi_{\text{ref}}(\tilde{U}(\tau)) - \nabla \log \pi_{\text{ref}}(U(\tau)) + 2 \nabla \log p_{T-\tau}(U(\tau)) - 2s_N(\tilde{U}(\tau), T - \tau) \rangle d\tau \\ &= |h_0|^2 + 2b^2 \int_0^t \langle h_\tau, \nabla \log \pi_{\text{ref}}(\tilde{U}(\tau)) - \nabla \log \pi_{\text{ref}}(U(\tau)) \rangle d\tau \\ &\quad + 4b^2 \int_0^t \langle h_\tau, \nabla \log p_{T-\tau}(U(\tau)) - \nabla \log p_{T-\tau}(\tilde{U}(\tau)) \rangle + \langle h_\tau, \nabla \log p_{T-\tau}(\tilde{U}(\tau)) - s_N(\tilde{U}(\tau), T - \tau) \rangle d\tau. \end{aligned}$$

Therefore, applying Assumptions 1 and 2, and Hölder's inequality we get

$$\mathbb{E}[|h_t|^2] \leq \mathbb{E}[|h_0|^2] + 2b^2 (C_{\text{ref}} - 2C_p) \int_0^t \mathbb{E}[|h_\tau|^2] d\tau + 4b^2 \int_0^t \mathbb{E}[|h_\tau|^2]^{\frac{1}{2}} \frac{C_e(T - \tau)}{Nr} d\tau,$$

where recall that the weighted samples and the SDE system are mutually independent. Applying (non-linear) Grönwall inequality (Dragomir, 2003, Theorem 21) we finally obtain

$$\mathbb{E}[|h_t|^2] \leq \mathbb{E}[|h_0|^2] e^{b^2 (C_{\text{ref}} - 2C_p)t} + 2b^2 N^{-r} \int_0^t C_e(T - \tau) e^{b^2 (C_{\text{ref}} - 2C_p)(t - \tau)} d\tau. \quad (19)$$

This directly implies an upper bound for the 2-Wasserstein distance between the measures of $U(t) \sim q_t$ and $\tilde{U}(t) \sim \tilde{q}_t$:

$$W_2^2(\tilde{q}_t, q_t) \leq e^{b^2 (C_{\text{ref}} - 2C_p)t} W_2^2(p_T, \pi_{\text{ref}}) + 2b^2 N^{-r} \bar{C}_e(t, T), \quad (20)$$

where $\bar{C}_e(t, T) := \int_0^t C_e(T - \tau) \exp[b^2 (C_{\text{ref}} - 2C_p)(t - \tau)] d\tau$.

C. Proof of Corollary 1

Recall the forward (Langevin) equation

$$\begin{aligned} dX(t) &= b^2 \nabla \log \pi_{\text{ref}}(X(t)) dt + \sqrt{2} b dW(t), \\ X(0) &\sim \pi. \end{aligned} \quad (21)$$

The error $W_1(p_t, \pi_{\text{ref}}) \rightarrow 0$ geometrically fast as $t \rightarrow \infty$, more specifically,

$$W_2^2(p_T, \pi_{\text{ref}}) \leq e^{-2b^2 C_{\text{ref}}^- T} W_2^2(\pi, \pi_{\text{ref}}), \quad (22)$$

where C_{ref}^- is the concave constant in Assumption 1. This is a classical result, see, for instance, Ambrosio et al. (2008) and von Renesse & Sturm (2005). Now choose $N^{r-c} = \bar{C}_e(t, T)$ for any constant c such that $0 < c < r$, and substitute Equation (22) into Equation (20) we get

$$W_2^2(\tilde{q}_t, q_t) \leq 2b^2 N^{-c} + e^{b^2 (C_{\text{ref}} - 2C_p)t - 2b^2 C_{\text{ref}}^- T} W_2^2(\pi, \pi_{\text{ref}}). \quad (23)$$

Recall that $t \mapsto C_e(t)$ is positive non-increasing, therefore $t \mapsto \bar{C}_e(t, T)$ is increasing for any $T > 0$. The aim here is to show there exists a sequence $t \mapsto T(t)$ such that $\lim_{t \rightarrow \infty} W_1(\tilde{q}_t, q_t) = 0$ *without further imposing conditions on C_e* . Clearly, the limit holds when 1) $T(t) > t$ uniformly and 2) $t \mapsto N(t)$ is increasing. To show 2), we take derivative of $t \mapsto \bar{C}_e(t, T(t))$ obtaining

$$\bar{C}_e'(t, T(t)) = C_e(T(t) - t) + \int_0^t T'(t) C_e'(T(t) - \tau) e^{z(t-\tau)} + C_e(T(t) - \tau) e^{z(t-\tau)} z d\tau,$$

where we shorthand $z = b^2 (C_{\text{ref}} - 2C_p)$ for simplicity. Using integration by parts on $\int_0^t C_e'(T(t) - \tau) e^{z(t-\tau)} d\tau$ we get

$$\int_0^t C_e'(T(t) - \tau) e^{z(t-\tau)} d\tau = -C_e(T(t) - t) + C_e(T(t)) e^{zt} - \int_0^t C_e(T(t) - \tau) e^{z(t-\tau)} z d\tau. \quad (24)$$

Therefore,

$$\bar{C}_e'(t, T(t)) = (1 - T'(t)) \left(C_e(T(t) - t) + \int_0^t C_e(T(t) - \tau) e^{z(t-\tau)} z d\tau \right) + T'(t) C_e(T(t)) e^{zt}. \quad (25)$$

Due to the positivity of C_e , a sufficient condition for the derivative above being positive is $T'(t) > 0$ and $T'(t) \leq 1$. The set of such sequence is not empty, and a trivial example is $T(t) = t + \delta$ for some parameter $\delta > 0$, which satisfies 1) and is clearly independent of any of the system components. Moreover, the exponent $b^2 (C_{\text{ref}} - 2C_p)t - 2b^2 C_{\text{ref}}^- T(t)$ is always negative under such an example.

Note that the factor N^{-c} is likely suboptimal since it can not exceed the importance sampling factor r . This is intuitive, because we are essentially asking N to compensate the SDE accumulation error. It is possible to relax from this and choose $c \geq r$. This in turn means that we need more assumptions on C_e to ensure that $\bar{C}_e(t, T(t))$ is *non-increasing* in t , in contrast to what we aimed here for being increasing which is mild. However, such settings will make Assumption 2 more difficult to verify in reality. Since making mild and practically checkable assumptions on score approximation remains an open

discussion (see, e.g., De Bortoli, 2022; Chen et al., 2023), we opt for a suboptimal bound under assumptions that are simple and easy-to-verify (n.b., we have only assumed C_e to be a positive non-increasing function without specifying any rate).

The result essentially reveals how we can leverage information from π_{ref} to achieve for lower resampling error. Commonly used resampling schemes assume that we only have access to the given samples $\{(w_i, X_i)\}_{i=1}^N$. On the contrary, diffusion resampling additionally assumes a sampler for π_{ref} that approximates π yielding more information of the target. The diffusion resampling effectively fuses the samples from $\{(w_i, X_i)\}_{i=1}^N$ and π_{ref} to obtain the re-samples, with the constant b indicating how much we rely on π_{ref} . With $b = 0$, it means that we completely trust the samples from π_{ref} , and this is indeed optimal when $\pi_{\text{ref}} = \pi$. See Appendix D for more details.

D. Elaboration of Remark 1

Recall the forward/noising process

$$dX(t) = b^2 \nabla \log \pi_{\text{ref}}(X(t)) dt + \sqrt{2} b dW(t), \quad X(0) \sim \pi, \quad (26)$$

which corresponds to the reversal

$$dU(t) = b^2 [-\nabla \log \pi_{\text{ref}}(U(t)) + 2 \nabla \log p_{T-t}(U(t))] dt + \sqrt{2} b dW(t), \quad U(0) \sim p_T. \quad (27)$$

Also recall the associated h -function $h(x, t) = \sum_{i=1}^N w_i p_{t|0}(x | X_i)$. Now if we instead initialise $X(0)$ at the empirical $\pi^N := \sum_{i=1}^N w_i \delta_{X_i}$, the resulting process $t \mapsto X^N(t)$ becomes the correspondence of reversal

$$dU^N(t) = b^2 [-\nabla \log \pi_{\text{ref}}(U^N(t)) + 2 \nabla \log h(U^N(t), T-t)] dt + \sqrt{2} b dW(t), \quad U^N(0) \sim p_T^N, \quad (28)$$

where $p_T^N(\cdot) = h(\cdot, T)$, and at time $t = T$ we recover $U^N(T) \sim \pi^N$. As such, we can see that this empirical forward-reversal pair is essentially a differentiable reparametrisation of multinomial resampling. However, we do not implement this reversal in practice, since sampling from the mixture p_T^N too typically introduces discrete randomness. The actually implementable reversal is

$$d\tilde{U}(t) = b^2 [-\nabla \log \pi_{\text{ref}}(\tilde{U}(t)) + 2 \nabla \log h(\tilde{U}(t), T-t)] dt + \sqrt{2} b dW(t), \quad \tilde{U}(0) \sim \pi_{\text{ref}}. \quad (29)$$

At time $t = T$, we obtain $\tilde{U}(T) \sim \sum_{i=1}^N \gamma_i \delta_{X_i}$, where $\gamma_i \neq w_i$ depending on π_{ref} . To arrive at the new weights, let us denote the transition distribution of U^N by $\tilde{q}_{t|s}^N$ which is the same for $\tilde{U}$. Equations (26) and (27) imply that $p_T^N(u_0) \tilde{q}_{T|0}^N(u_T | u_0) = \pi^N(u_T) p_{T|0}(u_0 | u_T)$. Therefore, the distribution of $\tilde{U}(T)$ is

$$\begin{aligned} \tilde{q}_T(u_T) &= \int \pi_{\text{ref}}(u_0) \tilde{q}_{T|0}^N(u_T | u_0) du_0 = \int \pi_{\text{ref}}(u_0) \tilde{q}_{T|0}^N(u_T | u_0) \frac{p_T^N(u_0)}{p_T^N(u_0)} du_0 \\ &= \int \frac{\pi_{\text{ref}}(u_0) p_{T|0}(u_0 | u_T)}{p_T^N(u_0)} \pi^N(u_T) du_0 = \sum_{i=1}^N \gamma_i(u_T) \delta_{X_i}(u_T), \end{aligned} \quad (30)$$

where the weight $\gamma_i(u_T) = w_i \int \frac{\pi_{\text{ref}}(u_0) p_{T|0}(u_0 | u_T)}{p_T^N(u_0)} du_0$ depends on the spatial location, and the integral is the Radon–Nikodym derivative between $\tilde{q}_T$ and π^N . Clearly, we see that the diffusion resampling is essentially an importance sampling upon π^N similar to soft resampling. However, the crucial difference is that the soft resampling’s importance proposal is completely uninformative about the target while the diffusion resampling is (i.e., the proposal π_{ref} and derivative γ_i know information about the target). As a special case, if we choose $\pi_{\text{ref}} = p_T^N$ then $\gamma_i = w_i$, reducing to the target empirical measure.

E. Error analysis of the resampling mapping

Previously in our main results (e.g., Corollary 1), we have analysed the error between the resampled distribution and the underlying true continuous distribution. In this section, we forgo the continuous distribution, and purely analyse the resampling mapping, that is, the L_2 error between the input and output ensembles.

Recall the input ensemble $\pi^N := \sum_{i=1}^N w_i \delta_{X_i}$, the resampled ensemble $\pi^{*,N}$, and denote $\pi^N(\phi) := \mathbb{E}_{\pi^N}[\phi] = \sum_{i=1}^N w_i \phi(X_i)$ for any test function ϕ . Define the χ^2 divergence by $\chi^2(p \| q) := \int (p(x) - q(x) / q(x)^2) q(x) dx$. Based on Equation (30) we have

$$\pi^{*,N}(\phi) = \int \pi_{\text{ref}}(u_0) \sum_{i=1}^N \phi(X_i) \frac{w_i p_{T|0}(u_0 | X_i)}{p_T^N(u_0)} du_0 = \int \pi_{\text{ref}}(u_0) \tilde{\phi}_N(u_0) du_0, \quad (31)$$

where $\tilde{\phi}_N(u_0) := \sum_{i=1}^N \phi(X_i) \frac{w_i p_{T|0}(u_0 | X_i)}{p_T^N(u_0)}$. Then $\pi^N(\phi) = \sum_{i=1}^N w_i \phi(X_i) \int p_{T|0}(u_0 | X_i) du_0 = \int p_T^N(u_0) \tilde{\phi}_N(u_0) du_0$ and we have the residual $\pi^{*,N}(\phi) - \pi^N(\phi) = \int (\pi_{\text{ref}}(u_0) - p_T^N(u_0)) \tilde{\phi}_N(u_0) du_0$. Hence

$$|\pi^{*,N}(\phi) - \pi^N(\phi)|^2 = \int \left| \tilde{\phi}_N(u_0) (\pi_{\text{ref}}(u_0) - p_T^N(u_0)) \frac{\sqrt{p_T^N(u_0)}}{\sqrt{p_T^N(u_0)}} \right|^2 du_0 \leq \pi^N(\phi^2) \chi^2(\pi_{\text{ref}} \| p_T^N), \quad (32)$$

where the identity $\tilde{\phi}_N(u_0)^2 \leq \sum_{i=1}^N \phi(X_i)^2 \frac{w_i p_{T|0}(u_0 | X_i)}{p_T^N(u_0)}$ was applied. Assume that the test function ϕ is uniformly bounded, i.e., $\pi^N(\phi^2) \leq c_\phi$, and also knowing that $p_T = \mathbb{E}[p_T^N]$, we finally have

$$\mathbb{E}[|\pi^{*,N}(\phi) - \pi^N(\phi)|^2] \leq c_\phi \mathbb{E}[\chi^2(p_T \| p_T^N)] + c_\phi \mathbb{E} \left[\int \frac{p_T(u) - \pi_{\text{ref}}(u)}{p_T^N(u)} du \right]. \quad (33)$$

F. Common experiment settings

Unless otherwise stated, all experiments share the following same settings.

Implementations are based on JAX (Bradbury et al., 2018). When applying a Wasserstein distance to quantify the quality of approximate samples, we use the earth-moving cost function denoted by W_1 , approximated by a sliced Wasserstein distance with 1,000 projections. We directly use the implementation by OTT-JAX (Cuturi et al., 2022). For neural network implementations, we use Flax (Heek et al., 2024).

For all particle filtering, the resampling is triggered at every steps. When applied to a state-space model, a bootstrap construction of the Feynman–Kac model is consistently used.

Whenever dealing with probability density evaluations, such as importance sampling, diffusion resampling, and Gaussian mixture, we always implement in the log domain.

Diffusion resampling uses the mean-reverting construction of the reference distribution π_{ref} as in Algorithm 3, and we set the diffusion coefficient $b^2 = \Sigma_N$. In practice, the diffusion coefficient should be calibrated, as Corollary 1 implies that there is likely an optimal b . Here we choose $b^2 = \Sigma_N$ just to simplify comparison, and this choice is not necessarily optimal. All SDE solvers are applied on evenly spaced time grids $0 = t_0 < t_1 < \dots < t_K = T$.

The numerical solver for SDEs impact the quality of diffusion resampling. In all experiments involving diffusion resampling, we test a combination of ways for solving the resampling SDE:

- Four SDE integrators. The commonly used Euler–Maruyama, and the two exponential integrators by Jentzen & Kloeden (2009) and Lord & Rougemont (2004), see Section 2.3 for formulae, and Tweedie’s formula (Ho et al., 2020, essentially DDPM). They are called by EM, Jentzen–Kloeden, Lord–Rougemont, and Tweedie, respectively in the later context.
- SDE and probability flow. The resampling SDE in Equation 4 can be solved also with a probability flow ODE (Song et al., 2021). We test both SDE and ODE versions.
- Diffusion time T and the number of discretisation steps K . We have mainly tested for $T = 1, 2, 3$ and $K = 4, 8, 32$.

The settings above result in at least 72 combinations, and it is not possible to report them all in the main body of the paper due to page limit. Therefore, in the main paper we only report the best combination, and we detail the rest of the combinations in appendix.

Gumbel-Softmax resampling The seminal work by Jang et al. (2017) did not formulate how the Gumbel trick can be applied for resampling. We here make it explicit. The gist is to approximate the discrete indexing of multinomial resampling by a matrix-vector multiplication which is differentiable. Recall our weighted samples $\{(w_i, X_i)\}_{i=1}^N$, and define a matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$ aggregating all the N samples. For each i , draw N independent samples $\{u_{i,j} \sim \text{Uniform}[0, 1]\}_{j=1}^N$ and compute $g_{i,j} = -\log \log u_{i,j}$. Then, the i -th re-sample is

$$X_i^* = \sum_{j=1}^N S_{i,j} X_j,$$

where $S_i \in \mathbb{R}^{1 \times N}$ is a Softmax vector with elements $S_{i,j} = \bar{S}_{i,j} / \sum_{j=1}^N \bar{S}_{i,j}$, where $\bar{S}_{i,j} = \exp((\log w_j + g_{i,j}) / \tau)$. This is independently repeated for $i = 1, 2, \dots, N$ to generate the re-samples $\{X_i^*\}_{i=1}^N$. Upon the limit $\tau \rightarrow 0$, this recovers multinomial resampling, but the variance of the gradient estimation will grow too. See also Rosato et al. (2022).

Soft resampling The approach by Karkus et al. (2018) view the samples $\{(w_i, X_i)\}_{i=1}^N$ as a discrete distribution with probability given by their normalised weights. Denote this discrete distribution by π^D . Directly sampling from π^D using weighted random choice will result in undefined gradients. Soft resampling mitigates this issue by introducing another proposal (discrete) distribution q^D defined by $q^D(X_i) := \alpha w_i + (1 - \alpha) \frac{1}{N}$ for $i = 1, 2, \dots, N$. Then, sampling from π^D can be achieved by an importance sampling as follows.

1. Draw $I_i \sim \text{Categorical}(w_1^q, w_2^q, \dots, w_N^q)$, where $w_i^q := q^D(X_i) = \alpha w_i + (1 - \alpha) \frac{1}{N}$.
2. Indexing $X_i^* = X_{I_i}$. Until here, we have sampled from the proposal q^D .
3. Weight $w_i^* = \pi^D(X_i^*) / q^D(X_i^*) = w_{I_i} / w_{I_i}^q$ and then normalise.
4. Return $\{(w_i^*, X_i^*)\}_{i=1}^N$.

Clearly, we can see that when $\alpha = 0$, the gradient is fully defined but the resampling and the gradient completely discard information from the weights $\{w_i\}_{i=1}^N$, resulting in high variance. The setting $\alpha = 1$ recovers the standard multinomial resampling, but the gradient becomes undefined. The gradient produced by method is thus always biased. Furthermore, we can also see that the soft resampling does not return uniform resampling weights unlike other methods, which to some extents, contradicts the purpose of resampling.

We will compare to the soft resampling and Gumbel-Softmax resampling with different settings of their tuning parameters.

G. Gaussian mixture resampling

Recall the model

$$\begin{aligned} \phi(x) &= \sum_{i=1}^c \omega_i \mathcal{N}(x; m_i, v_i), \\ p(y | x) &= \mathcal{N}(y; H x, \Xi), \end{aligned} \tag{34}$$

where $x \in \mathbb{R}^d$, $y \in \mathbb{R}$, $d = 8$, and the number of components $c = 5$. The posterior distribution $\pi(x) \propto \phi(x) p(y | x)$ is also a Gaussian mixture $\pi(x) = \sum_{i=1}^c \Omega_i \mathcal{N}(x; \mathcal{M}_i, \mathcal{V}_i)$ (Zhao, 2025), given by

$$\begin{aligned} G_i &= H v_i H^\top + \Xi, \\ \bar{\Omega}_i &= \omega_i \mathcal{N}(y; H m_i, G_i), \\ \Omega_i &= \bar{\Omega}_i / \sum_{j=1}^c \bar{\Omega}_j, \\ \mathcal{M}_i &= m_i + v_i H^\top G_i^{-1} (y - H m_i), \\ \mathcal{V}_i &= v_i - v_i H^\top G_i^{-1} H v_i. \end{aligned}$$

Table 3. Sliced 1-Wasserstein distance (SWD, scaled by 10^{-1}) of the Gaussian mixture experiments. This table focuses only on the diffusion resampling. We see that the Jentzen–Kloeden integrator performs the best in average.

Method	Euler–Maruyama		Lord–Rougemont		Jentzen–Kloeden		Tweedie
	ODE	SDE	ODE	SDE	ODE	SDE	SDE
$T = 1, K = 8$	1.92 ± 0.41	2.39 ± 0.57	1.79 ± 0.27	3.04 ± 0.67	1.64 ± 0.35	2.70 ± 0.62	2.10 ± 0.51
$T = 1, K = 32$	1.31 ± 0.30	1.03 ± 0.26	1.29 ± 0.30	1.03 ± 0.26	1.26 ± 0.30	1.03 ± 0.26	1.01 ± 0.26
$T = 1, K = 128$	1.23 ± 0.31	0.92 ± 0.25	1.23 ± 0.31	0.90 ± 0.25	1.22 ± 0.31	0.91 ± 0.25	0.92 ± 0.25
$T = 2, K = 8$	2.45 ± 0.56	4.15 ± 0.91	4.41 ± 0.74	7.38 ± 1.37	1.93 ± 0.45	5.56 ± 1.04	3.28 ± 0.80
$T = 2, K = 32$	1.02 ± 0.22	1.38 ± 0.34	1.22 ± 0.23	1.66 ± 0.39	0.93 ± 0.22	1.51 ± 0.35	1.27 ± 0.32
$T = 2, K = 128$	0.82 ± 0.21	0.84 ± 0.24	0.84 ± 0.21	0.85 ± 0.24	0.82 ± 0.21	0.85 ± 0.24	0.83 ± 0.24
$T = 3, K = 8$	3.22 ± 0.73	5.60 ± 1.14	10.78 ± 2.17	13.21 ± 2.14	2.53 ± 0.60	8.62 ± 1.42	4.12 ± 0.97
$T = 3, K = 32$	1.20 ± 0.26	1.89 ± 0.46	2.27 ± 0.45	2.52 ± 0.57	1.06 ± 0.24	2.16 ± 0.49	1.66 ± 0.43
$T = 3, K = 128$	0.82 ± 0.21	0.89 ± 0.24	0.93 ± 0.21	0.94 ± 0.25	0.80 ± 0.21	0.91 ± 0.25	0.87 ± 0.24

Table 4. Sliced 1-Wasserstein distance (SWD, scaled by 10^{-1}) and resampling variance (scaled by 10^{-2}) of the Gaussian mixture experiments. This table should be compared to Table 3.

Method	SWD	Resampling variance	Method	SWD	Resampling variance	Method	SWD	Resampling variance
OT ($\varepsilon = 0.3$)	0.84 ± 0.22	3.42 ± 3.26	Gumbel 0.2	1.40 ± 0.24	3.92 ± 3.74	Soft 0.2	0.92 ± 0.25	4.71 ± 3.60
OT ($\varepsilon = 0.6$)	0.97 ± 0.21	3.41 ± 3.29	Gumbel 0.2	2.52 ± 0.41	3.90 ± 3.76	Soft 0.4	0.86 ± 0.23	4.11 ± 3.98
OT ($\varepsilon = 0.8$)	1.08 ± 0.20	3.42 ± 3.30	Gumbel 0.4	5.16 ± 0.82	3.83 ± 3.76	Soft 0.8	0.85 ± 0.24	4.12 ± 4.09
OT ($\varepsilon = 0.9$)	1.14 ± 0.20	3.42 ± 3.29	Gumbel 0.8	11.57 ± 1.81	3.59 ± 3.54	Soft 0.9	0.83 ± 0.24	3.75 ± 3.77

At each individual experiment, we randomly generate the Gaussian mixture components. Specifically, we draw the mixture mean $m_i \sim \text{Uniform}([-5, 5]^d)$ and the covariance $v_i = \bar{v}_i + I_d$, where $\bar{v}_i \sim \text{Wishart}(I_d)$. To reduce experiment variance, the weights $\omega_i = 1/c$ are fixed to be even, the observation operator $H \in \mathbb{R}^{1 \times d}$ is an all-one vector, and $\Xi = 1$. Experiments are repeated 100 times independently.

Remark 3 (Resampling variance). *In literature, the resampling variance is usually defined as the variance of the resampling algorithm itself conditioned on the input samples, that is, $\text{Var} \left[\frac{1}{N} \sum_{i=1}^N X_i^* \mid \{(w_i, X_i)\}_{i=1}^N \right]$. This is useful to heuristically gauge the noise level of resampling. However, this does not inform how well the re-samples approximate the true distribution π . With a slight abuse of terminology, we define the resampling variance as the mean estimator error $\mathbb{E} \left[\left(\frac{1}{N} \sum_{i=1}^N X_i^* - \mathcal{M} \right)^2 \right]$, where $\mathcal{M}$ stands for the true mean of the Gaussian mixture posterior π . The expectation is approximated by the 100 independent Monte Carlo runs.*

Tables 3 and 4 show more detailed results compared to Table 1 in the main body. We see that with a fixed fine-enough discretisation, the SWD in general decreases as T increases independent of the SDE integrator used. It also shows that the probability flow (ODE) version for solving the resampling SDE is better than SDE, especially when K is small. As for the integrators, Jentzen–Kloeden appears to be the best, and it is especially more useful when the discretisation is coarse. The OT resampling performs roughly the same as with diffusion resampling with setting $\varepsilon = 0.8$ and $T = 3, K = 32$.

H. Time comparison

In this experiment we focus on the actual computational time and compare different methods when controlling them to have a similar level of estimation error. The main result is shown in Figure 1, supplemented by Tables 5 and 6.

The results are averaged over 50 independent runs on an NVIDIA A100 80G GPU with a fixed dimension $d = 8$. We have also experimented on a CPU (AMD EPYC 9354 32-Core) but we did not observe any significant difference compared to that of GPU worth to report.

For previous experiments, T and K are given, and the time interval is adapted by $\Delta = T / (K + 1)$. For this experiment, we fix a time interval $\Delta = 0.1$ and change K , so that $T = \Delta (K + 1)$. We compute the resampling error which is a square root of the resampling variance.

Tables 5 and 6 show that the diffusion and OT methods have a similar estimation error especially for large sample size

Table 5. The resampling error of diffusion resampling in terms of the number of time steps K and sample size N . Related to the time experiment in Appendix H.

Method	Number of samples N						
	128	256	512	1024	2048	4096	8192
$K = 4$	0.29 ± 0.07	0.21 ± 0.04	0.15 ± 0.04	0.11 ± 0.03	0.08 ± 0.02	0.05 ± 0.01	0.04 ± 0.01
$K = 8$	0.28 ± 0.06	0.21 ± 0.04	0.15 ± 0.04	0.11 ± 0.03	0.08 ± 0.03	0.05 ± 0.01	0.04 ± 0.01
$K = 16$	0.27 ± 0.06	0.20 ± 0.04	0.15 ± 0.04	0.11 ± 0.03	0.08 ± 0.03	0.04 ± 0.01	0.04 ± 0.01
$K = 32$	0.27 ± 0.06	0.20 ± 0.04	0.15 ± 0.04	0.11 ± 0.03	0.08 ± 0.03	0.04 ± 0.01	0.04 ± 0.01

 Table 6. The resampling error of OT in terms of entropy regularisation ε and sample size N . Related to the time experiment in Appendix H.

Method	Number of samples N						
	128	256	512	1024	2048	4096	8192
$\varepsilon = 0.8$	0.27 ± 0.06	0.20 ± 0.03	0.15 ± 0.03	0.11 ± 0.02	0.08 ± 0.02	0.05 ± 0.01	0.04 ± 0.01
$\varepsilon = 0.4$	0.27 ± 0.06	0.20 ± 0.03	0.15 ± 0.03	0.11 ± 0.02	0.08 ± 0.02	0.05 ± 0.01	0.04 ± 0.01
$\varepsilon = 0.2$	0.27 ± 0.06	0.20 ± 0.03	0.15 ± 0.03	0.11 ± 0.02	0.08 ± 0.02	0.05 ± 0.01	0.04 ± 0.01
$\varepsilon = 0.1$	0.27 ± 0.06	0.20 ± 0.03	0.15 ± 0.03	0.11 ± 0.02	0.08 ± 0.02	0.05 ± 0.01	0.04 ± 0.01

N . Moreover, the results empirically verify the theoretical connection between the OT regularisation ε and the diffusion time T stated in Section 3. That is, K scales proportionally to $1/\varepsilon$ in order for them to have the same level of statistical performance. However, as evidenced in Figure 1, the diffusion resampling is in average faster than that of OT. When both methods are at their best estimation performance (i.e., $\varepsilon = 0.1$ and $K = 32$), diffusion resampling is still faster than OT.

I. Linear Gaussian SSM

Recall the model

$$\begin{aligned} Z_j \mid Z_{j-1} &\sim \mathcal{N}(z_j; \theta_1 z_{j-1}, I_d), \quad Z_0 \sim \mathcal{N}(0, I_d), \\ Y_j \mid Z_j &\sim \mathcal{N}(y_j; \theta_2 z_j, 0.5 I_d), \end{aligned} \quad (35)$$

where we set parameters $\theta_1 = 0.5$ and $\theta_2 = 1$. For each experiment, we generate a measurement sequence with $j = 0, 1, \dots, 128$ steps, with $N = 32$ particles. The loss function estimate error $\|L - \hat{L}\|_2^2 := \int_{\Theta} (L(\theta_1, \theta_2) - \hat{L}(\theta_1, \theta_2))^2 d\theta_1 d\theta_2$, where $\hat{L}$ is estimated by Trapezoidal quadrature at the Cartesian grids $\Theta = [\theta_1 - 0.1, \theta_1 + 0.1] \times [\theta_2 - 0.1, \theta_2 + 0.1]$. The filtering error is evaluated by Kullback–Leibler (KL) divergence between the true filtering distribution (which is a Gaussian computed exactly by a Kalman filter) and the particle filtering samples. Precisely, the error is defined by $\frac{1}{129} \sum_{j=0}^{128} \text{KL}(\mathcal{N}(m_j^f, V_j^f) \parallel \mathcal{N}(\hat{m}_j^f, \hat{V}_j^f))$, where $\mathcal{N}(m_j^f, V_j^f)$ is the true filtering distribution at step j , and $\mathcal{N}(\hat{m}_j^f, \hat{V}_j^f)$ stands for the empirical approximation by the particle samples.

The optimiser L-BFGS is implemented using JAXopt (Blondel et al., 2021) `scipyminimize` wrapper with default settings and initial parameters $\theta + 1$.

We observe a numerical issue due to the gradient estimate by the resampling methods. The L-BFGS optimiser frequently diverges. This is expected, since L-BFGS is highly sensitive to the quality of gradient estimate (Xie et al., 2020); see also Zhao & Sarmarvuori (2023) for an empirical validation. As such, when we report the results we have defined a convergent run by 1) positive convergence flag returned by the L-BFGS optimiser and 2) the error $\|\theta - \hat{\theta}\|_2 < 1.9$ is not absurd. It turns out that the effect is especially pronounced for the soft and Gumbel-Softmax resampling, where at 100 runs, only nearly 10% return a successful flag of the optimiser, and all their estimated parameters diverge to a meaningless position. This was not problematic for the diffusion and OT samplers, where they have approximately 80% success rate in average.

We also observe a numerical issue of OT implementation. By default, for instance, in OTT-JAX, gradient propagation through the Sinkhorn solver uses implicit differentiation by solving a linear system. This makes gradient computation of the OT resampling efficient. However, the linear system often becomes ill-conditioned for most runs, and making OT parameter

Table 7. The errors of loss function estimate $\|L - \hat{L}\|_2$ associated with the linear Gaussian state-space model (LGSSM) experiment. This table focuses only on the diffusion resampling and should be compared to Table 10

Method	Euler–Maruyama		Lord–Rougemont		Jentzen–Kloeden		Tweedie
	ODE	SDE	ODE	SDE	ODE	SDE	SDE
$T = 1, K = 4$	2.72 ± 2.12	2.66 ± 2.11	2.90 ± 2.24	2.63 ± 2.10	2.72 ± 2.12	2.61 ± 2.08	2.69 ± 2.13
$T = 1, K = 8$	2.72 ± 2.13	2.70 ± 2.00	2.77 ± 2.17	2.69 ± 1.98	2.72 ± 2.13	2.68 ± 1.98	2.71 ± 2.02
$T = 1, K = 16$	2.72 ± 2.13	2.77 ± 2.23	2.74 ± 2.15	2.79 ± 2.23	2.72 ± 2.13	2.77 ± 2.23	2.77 ± 2.23
$T = 1, K = 32$	2.72 ± 2.14	2.67 ± 2.12	2.73 ± 2.15	2.66 ± 2.11	2.72 ± 2.14	2.66 ± 2.11	2.67 ± 2.12
$T = 2, K = 4$	2.72 ± 2.11	2.58 ± 2.06	5.68 ± 4.25	2.55 ± 2.05	2.72 ± 2.11	2.51 ± 1.95	2.64 ± 2.14
$T = 2, K = 8$	2.72 ± 2.12	2.66 ± 1.93	3.29 ± 2.51	2.69 ± 1.92	2.72 ± 2.12	2.62 ± 1.89	2.70 ± 1.97
$T = 2, K = 16$	2.72 ± 2.13	2.79 ± 2.31	2.87 ± 2.23	2.81 ± 2.30	2.72 ± 2.13	2.78 ± 2.32	2.80 ± 2.30
$T = 2, K = 32$	2.72 ± 2.13	2.62 ± 2.10	2.77 ± 2.18	2.62 ± 2.10	2.72 ± 2.13	2.61 ± 2.09	2.62 ± 2.11
$T = 3, K = 4$	2.72 ± 2.11	2.51 ± 2.01	14.89 ± 8.90	33.38 ± 5.45	2.72 ± 2.11	2.57 ± 1.67	2.63 ± 2.13
$T = 3, K = 8$	2.72 ± 2.11	2.61 ± 1.92	5.90 ± 4.39	2.70 ± 2.04	2.72 ± 2.11	2.55 ± 1.89	2.67 ± 1.96
$T = 3, K = 16$	2.72 ± 2.12	2.78 ± 2.33	3.37 ± 2.56	2.82 ± 2.33	2.72 ± 2.12	2.76 ± 2.35	2.80 ± 2.32
$T = 3, K = 32$	2.72 ± 2.13	2.58 ± 2.08	2.89 ± 2.23	2.56 ± 2.09	2.72 ± 2.13	2.56 ± 2.07	2.59 ± 2.09

estimation diverges largely. We thus had to disable this feature by unrolling the gradients. To fairly compare SDEs and OT, we have also unrolled gradient propagation through SDEs without using any implicit/adjoint methods.

The results are detailed in Tables 7 to 10. We clearly see that the diffusion resampling outperforms other methods across all the three metrics, in particular for the filtering and parameter estimation errors. The superiority of the log-likelihood function estimation is not significant due to that the loss function’s magnitude does not change much in Θ , see also Figure 2.

Let us focus on the diffusion resampling in Tables 7 and 9. In Tables 7 we find that the loss function estimation error seems to be invariant in the time T and steps K when using an ODE solver (except for Lord–Rougemont), giving a relatively large error 2.72. With a fixed T , it is not clear if increasing K will improve the log-likelihood estimate. This also resonates with Table 9 that increasing K can possibly lead to worst even exploding parameter estimation. This may be caused by numerical errors when back-propagating gradients through SDE solvers which can be addressed by, for instance, Kidger et al. (2021). On the other hand, with K fixed, increasing T improves the result. Like in the previous experiments, the Jentzen–Kloeden exponential integrator consistently achieves the best result, albeit marginally, while Lord–Rougemont is the most sensitive to K .

Table 8 shows the performance of the filtering, and this does not compute any gradients. We see that improving T and K improves the filtering estimation in general. This empirically verifies the results from Tables 7 and 9 that efficient and accurate back-propagation through SDE solvers may be necessary.

Figure 2 shows loss function landscapes estimated by the particle filtering ($N = 32$ particles) with different resampling schemes. The calibration parameters for Gumbel-Softmax and soft resampling are 0.2 and 0.1, respectively, and for diffusion, we use Jentzen–Kloeden SDE integrator with $T = 2$ and $K = 4$, and for OT $\varepsilon = 0.3$. We see in this figure that multinomial gives the most noisy estimate, hard for searching the optimum. This is also true for soft resampling, and even at a low parameter 0.1 which already means very uninformative resampling, the loss function is still rather noisy. On the other hand, the loss function estimates by diffusion, OT, and Gumbel-Softmax look smooth.

J. Prey-predator model

Recall our prey-predator (or also called Lotka–Volterra) model

$$\begin{aligned}
 dC(t) &= C(t) (\alpha - \beta R(t)) dt + \sigma C(t) dW_1(t), \\
 dR(t) &= R(t) (\zeta C(t) - \gamma) dt + \sigma R(t) dW_2(t), \\
 Y_j &\sim \text{Poisson}(\lambda(C(t_j), R(t_j))),
 \end{aligned} \tag{36}$$

Table 8. The filtering error in terms of KL divergence (scaled by 10^{-1}) associated with the linear Gaussian state-space model (LGSSM) experiment. This table focuses only on the diffusion resampling and should be compared to Table 10

Method	Euler–Maruyama		Lord–Rougemont		Jentzen–Kloeden		Tweedie
	ODE	SDE	ODE	SDE	ODE	SDE	SDE
$T = 1, K = 4$	5.03 ± 5.75	5.05 ± 7.00	5.39 ± 6.57	5.03 ± 7.59	5.02 ± 5.73	4.94 ± 6.92	5.28 ± 8.24
$T = 1, K = 8$	4.99 ± 5.67	5.05 ± 7.69	5.10 ± 5.86	4.90 ± 6.95	4.99 ± 5.65	4.97 ± 7.36	5.10 ± 7.98
$T = 1, K = 16$	4.98 ± 5.68	5.11 ± 7.90	4.99 ± 5.64	5.50 ± 11.01	4.97 ± 5.67	5.17 ± 8.46	5.03 ± 7.27
$T = 1, K = 32$	5.01 ± 5.87	4.84 ± 5.24	5.00 ± 5.83	4.82 ± 5.26	5.00 ± 5.87	4.80 ± 5.13	4.85 ± 5.25
$T = 2, K = 4$	5.07 ± 5.84	4.71 ± 5.76	7.48 ± 9.61	5.06 ± 12.06	5.06 ± 5.82	4.38 ± 5.47	5.07 ± 6.81
$T = 2, K = 8$	5.03 ± 5.75	4.60 ± 5.20	5.87 ± 7.56	4.35 ± 4.49	5.02 ± 5.72	4.40 ± 4.78	4.82 ± 6.01
$T = 2, K = 16$	4.99 ± 5.66	5.24 ± 8.62	5.27 ± 6.18	5.18 ± 8.12	4.98 ± 5.65	5.17 ± 8.38	5.32 ± 8.97
$T = 2, K = 32$	4.98 ± 5.68	4.86 ± 5.48	5.01 ± 5.62	4.82 ± 5.32	4.97 ± 5.67	4.77 ± 5.28	4.89 ± 5.56
$T = 3, K = 4$	5.09 ± 5.89	4.39 ± 5.08	15.80 ± 19.50	3.59 ± 5.97	5.08 ± 5.87	4.13 ± 7.93	4.85 ± 5.71
$T = 3, K = 8$	5.06 ± 5.81	4.51 ± 4.92	7.68 ± 9.93	4.16 ± 4.35	5.05 ± 5.78	4.26 ± 4.49	4.76 ± 5.57
$T = 3, K = 16$	5.01 ± 5.71	5.17 ± 7.26	5.96 ± 7.74	5.29 ± 7.74	5.00 ± 5.69	5.17 ± 7.50	5.18 ± 7.31
$T = 3, K = 32$	4.98 ± 5.65	4.78 ± 5.41	5.26 ± 6.13	4.74 ± 5.39	4.97 ± 5.64	4.76 ± 5.44	4.81 ± 5.48

Table 9. The parameter estimation error (scaled by 10^{-1}) associated with the linear Gaussian state-space model (LGSSM) experiment. This table focuses only on the diffusion resampling and should be compared to Table 10.

Method	Euler–Maruyama		Lord–Rougemont		Jentzen–Kloeden		Tweedie
	ODE	SDE	ODE	SDE	ODE	SDE	SDE
$T = 1, K = 4$	1.32 ± 0.70	1.45 ± 1.90	1.72 ± 0.98	1.65 ± 1.61	1.36 ± 0.74	1.28 ± 0.70	1.46 ± 1.94
$T = 1, K = 8$	1.48 ± 0.93	1.70 ± 1.86	1.56 ± 0.94	2.25 ± 3.09	1.44 ± 0.82	2.14 ± 3.35	1.75 ± 2.51
$T = 1, K = 16$	1.76 ± 2.11	11.30 ± 5.22	1.59 ± 1.55	11.94 ± 4.90	1.40 ± 0.74	12.07 ± 4.68	11.93 ± 4.94
$T = 1, K = 32$	4.43 ± 5.39	14.14 ± 0.00	4.05 ± 5.33	14.14 ± 0.00	3.23 ± 4.38	14.14 ± 0.00	14.14 ± 0.00
$T = 2, K = 4$	1.34 ± 0.70	1.25 ± 0.70	6.07 ± 2.57	3.19 ± 3.83	1.33 ± 0.69	1.49 ± 1.55	1.26 ± 0.67
$T = 2, K = 8$	1.36 ± 0.76	1.30 ± 0.68	2.29 ± 1.22	1.68 ± 1.47	1.34 ± 0.70	1.29 ± 0.78	1.23 ± 0.62
$T = 2, K = 16$	1.39 ± 0.73	1.84 ± 2.52	1.80 ± 1.94	1.72 ± 2.33	1.48 ± 0.93	1.39 ± 0.89	1.63 ± 1.77
$T = 2, K = 32$	1.59 ± 1.69	10.57 ± 5.68	1.68 ± 2.05	12.40 ± 4.41	1.62 ± 1.67	11.49 ± 5.06	13.31 ± 3.06
$T = 3, K = 4$	1.34 ± 0.70	1.27 ± 0.74	8.74 ± 3.94	14.14 ± 0.00	1.39 ± 0.81	1.79 ± 1.09	1.23 ± 0.67
$T = 3, K = 8$	1.32 ± 0.70	1.25 ± 0.62	5.96 ± 2.64	2.26 ± 2.61	1.32 ± 0.70	1.28 ± 0.75	1.28 ± 0.61
$T = 3, K = 16$	1.40 ± 0.72	1.48 ± 1.56	2.48 ± 1.31	1.41 ± 0.89	1.39 ± 0.72	1.26 ± 0.74	1.53 ± 2.01
$T = 3, K = 32$	1.39 ± 0.73	1.95 ± 2.88	1.92 ± 2.07	3.20 ± 4.33	1.41 ± 0.76	3.26 ± 4.43	2.81 ± 4.32

Table 10. The errors of loss function, filtering in terms of KL divergence (scaled by 10^{-1}), and parameter estimation (scaled by 10^{-1}) associated with the linear Gaussian state-space model (LGSSM) experiment. This table focuses only on methods other than the diffusion resampling, and should be compared to Tables 7, 8, and 9.

Method	$\ L - \hat{L}\ _2$	Filtering KL	$\ \theta - \hat{\theta}\ _2$
OT ($\varepsilon = 0.4$)	2.64 ± 2.13	5.07 ± 6.21	1.53 ± 1.16
OT ($\varepsilon = 0.8$)	2.68 ± 2.16	5.07 ± 5.70	1.58 ± 1.22
OT ($\varepsilon = 1.6$)	2.76 ± 2.20	5.11 ± 5.17	1.49 ± 0.97
Gumbel (0.1)	2.79 ± 2.14	4.83 ± 5.76	NaN
Gumbel (0.3)	2.75 ± 2.17	4.89 ± 5.42	NaN
Gumbel (0.5)	2.73 ± 2.22	5.10 ± 5.93	NaN
Soft (0.5)	3.63 ± 2.10	7.75 ± 11.19	NaN
Soft (0.7)	3.08 ± 1.88	5.34 ± 7.82	NaN
Soft (0.9)	2.85 ± 1.80	4.66 ± 5.68	NaN
Multinomial	2.80 ± 1.84	5.49 ± 6.87	NaN

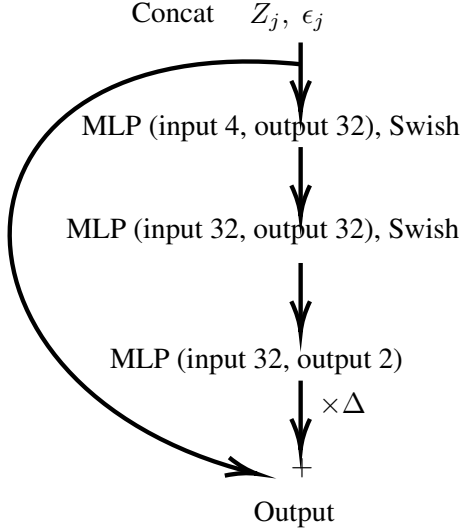


Figure 7. The neural network used for learning the prey-predator model. At the input, Z_j and ϵ are concatenated to a four-dimensional vector.

Table 11. Prediction error (RMSE) and the number of successful runs (out of 20) for the prey-predator model experiment.

Method	RMSE	Number of success
OT ($\epsilon = 0.3$)	2.35 ± 0.95	19
OT ($\epsilon = 0.5$)	2.72 ± 1.60	20
OT ($\epsilon = 1.0$)	3.27 ± 2.71	20
OT ($\epsilon = 1.5$)	3.52 ± 3.65	20
Gumbel (0.1)	9.05 ± 8.65	15
Gumbel (0.3)	2.09 ± 1.13	20
Gumbel (0.5)	2.40 ± 1.05	20
Soft (0.5)	3.29 ± 4.83	17
Soft (0.7)	2.13 ± 0.87	17
Soft (0.9)	1.89 ± 0.64	16
Stopped	1.96 ± 0.55	20

where we set $\alpha = \gamma = 6$, $\beta = 2$, $\zeta = 4$, and $\sigma = 0.15$. The observation rate function $\lambda: \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}_{\geq 0}^2$ is defined by

$$\lambda(c, r) := \frac{5}{1 + \exp\left(\begin{bmatrix} -5c \\ -cr \end{bmatrix} + 4\right)}. \quad (37)$$

We simulate the model with Milstein’s method and generate data at $t \in [0, 3]$ with 256 discretisation steps. The discretisation results in an SSM $Z_{j+1} = f(Z_j, \epsilon_j)$, where $Z_j \in \mathbb{R}^2$ encodes $C(t_j)$ and $R(t_j)$, and $\epsilon_j \in \mathbb{R}^2$ stands for Brownian motion increment. We train a neural network to approximate the SDE dynamics by minimising the negative log-likelihood, estimated by a particle filter with resampling. The neural network is simply a three-layers fully connected network with residual connection, see Figure 7, mimicking a discrete SDE solver. No prior knowledge of the dynamics is incorporated into the neural network. We train the neural network at a fixed 1,000 iterations by the Adam optimiser with learning rate 0.005. At testing, we make 100 independent predictions from the learnt model and compute the root mean square error (RMSE) with respect to a reference trajectory generated using the true dynamics. We use 64 particles, and repeat the experiments 20 times.

Results are shown in Tables 11 and 12. Since learning this model is challenging, most methods have experienced divergent runs. Notably, the diffusion resampling exhibits significantly less divergence, nearly a factor of two lower than the other methods. Moreover, the prediction error with diffusion resampling is substantially superior than the other methods; approximately two to five times better. This table is also consistent with that of the LGSSM experiment that the exponential integrators may not propagate SDE gradients effectively, except for Tweedie.

K. Vision-based pendulum dynamics tracking

Experiment settings We assume that the pendulum dynamics evolve according to Equation (16), with parameters $g = 9.81$ and $l = 0.4$. Using the known dynamics and the observation model specified in Equation (16) with observation noise $\sigma_{\text{obs}} = 0.01$, we generate a single sequence of observational data over $t \in [0, 4]$ using 256 discretisation steps (which is $j = 0, 1, \dots, 256$ in discrete time) and starting from the initial state $Z_0 = [\pi/4 \ 0]$. During training, we jointly optimise the two neural networks, f_θ and r_ϕ , parameterising the transition function and the decoder, respectively, by minimising the negative log-likelihood (NLL) estimated by the particle filter.

For completeness, we consider two experiment settings: in the first setting we employ effective sample size (ESS)-based adaptive resampling under a tempered observation likelihood. Although the tempering modifies the original model, it is a common practice in deep learning for training complex models (see, e.g., Ng et al., 2025). Here, the process noise covariance is fixed by $\Lambda_\zeta = \sigma_\zeta^2 I_2$, where $\sigma_\zeta^2 = 0.01$. We use $\sigma_{\text{train}} = 10 \sigma_{\text{obs}}$ and scale the observation log-likelihood. In particular,

Table 12. Prediction RMSE (first table) of diffusion resampling for the prey-predator model, and the associated number of successful runs (second table) out of 20. By comparing to Table 11 we see that all the entries here are largely better than the other resampling methods. However, we also see that exponential integrators do not always provide stable gradient back-propagation, although they provide accurate forward estimation.

Method	Euler–Maruyama		Lord–Rougemont		Jentzen–Kloeden		Tweedie
	ODE	SDE	ODE	SDE	ODE	SDE	SDE
$T = 1, K = 4$	1.29 ± 0.79	1.30 ± 0.98	NaN	17.01 ± 3.16	1.31 ± 0.79	2.22 ± 0.72	1.17 ± 0.75
$T = 1, K = 8$	1.31 ± 0.71	2.38 ± 6.56	16.62 ± 6.42	10.21 ± 7.42	1.29 ± 0.70	1.65 ± 0.90	1.23 ± 0.71
$T = 1, K = 16$	1.40 ± 0.89	1.95 ± 1.10	10.79 ± 10.42	5.78 ± 6.15	1.38 ± 0.88	5.20 ± 4.62	2.24 ± 1.02
$T = 2, K = 4$	1.21 ± 0.81	1.85 ± 1.22	NaN	NaN	1.21 ± 0.76	NaN	1.18 ± 0.76
$T = 2, K = 8$	1.26 ± 0.70	1.61 ± 0.79	NaN	13.88 ± 10.80	1.24 ± 0.72	2.05 ± 1.19	1.17 ± 0.68
$T = 2, K = 16$	1.29 ± 0.70	1.01 ± 0.47	9.52 ± 1.97	15.38 ± 13.57	1.30 ± 0.75	1.72 ± 1.04	1.29 ± 0.87

Method	Euler–Maruyama		Lord–Rougemont		Jentzen–Kloeden		Tweedie
	ODE	SDE	ODE	SDE	ODE	SDE	SDE
$T = 1, K = 4$	20	18	0	4	20	9	20
$T = 1, K = 8$	20	20	2	16	20	18	20
$T = 1, K = 16$	20	18	15	19	20	18	17
$T = 2, K = 4$	20	12	0	0	20	0	20
$T = 2, K = 8$	20	16	0	8	20	10	20
$T = 2, K = 16$	20	18	3	15	20	15	20

we compute the log-likelihood as the mean over pixel dimensions (rather than the sum) and rescale the result by a factor κ^{-1} (where $\kappa = 10^4$). We use a resampling threshold such that we resample if the ESS is less than $N = 32$. We train the neural networks for 3000 iterations. The learnable parameters are updated using the Adam optimiser (learning rate 10^{-4}) with global gradient clipping (maximum norm 1.0) and $\beta_2 = 0.999$ (other Adam defaults as in Optax). We use $N = 32$ particles to approximate the negative log-likelihood during training.

In the second setting, we consider a more challenging regime in which we force resampling at every filtering step after $j > 10$. Here, we use process noise with $\Lambda_\zeta = \sigma_\zeta^2 \text{diag}(0, 1)$ with $\sigma_\zeta^2 = 0.1$, and make an additive noise assumption in the learned transition model. We use $\sigma_{\text{train}} = 5 \sigma_{\text{obs}}$ and no other scaling of the log-likelihood compared to the true observation model. We train the neural networks for 1500 iterations. The learnable parameters are updated using the Adam optimiser (learning rate 2×10^{-4}) with global gradient clipping (maximum norm 1.0) and $\beta_2 = 0.99$ (other Adam defaults as in Optax). Here, we use $N = 16$ particles. The optimiser omits at most 10 consecutive non-finite parameter updates by leaving the current parameter values unchanged.

Results and evaluation We evaluate the learnt models by comparing the average SSIM and PSNR of image sequences obtained by unrolling the dynamics from Z_0 using the learnt transition function f_θ , and then generating images by passing the resulting state trajectory through the learnt decoder r_ϕ . All results corresponding to the first experiment setting are averaged over 5 independent runs, and all results corresponding to the second experiment setting are averaged over 9 independent runs.

For the first setting, results for different configurations of the diffusion resampler are presented in Table 13. Results for different configurations of the baselines are presented in Table 14. For the second, more challenging setting, in which resampling is invoked at nearly every filtering step, the corresponding results are presented in Tables 15 and 16, respectively. Figure 8 shows the median loss evolution during training for the best model configuration (in terms of mean SSIM and PSNR) from each resampling class in each experiment setting. Figure 9 shows mean prediction SSIM/PSNR for the second setting; see Figure 5 for a comparison to the corresponding results in the first setting.

The difference in SSIM/PSNR between the two experiments indicate that the second setting is more challenging for all resampling methods. In the first setting, performance is competitive, and diffusion resampling provides a stable end-to-end optimisation and effective integration into the high-dimensional learning pipeline. Notably, resampling is not always necessary for convergence in end-to-end training using particle filters, and prior work has reported cases where omitting resampling can be more effective (see, e.g., Karkus et al., 2018). In our experiments, we consider end-to-end training

Table 13. Prediction quality in terms of structural similarity index (SSIM) and PSNR (higher the better) for the pendulum dynamics tracking experiment, with respect to models trained using diffusion resampling in the first experiment setting. Each result is presented in terms of mean over 5 independent runs, together with the corresponding standard deviation, recorded after 3000 training iterations. For two configurations only 1 out of 5 (*) and 3 out of 5 (**) runs, respectively, were non-divergent.

Method	Euler–Maruyama				Jentzen–Kloeden				Tweedie	
	SDE		ODE		SDE		ODE		SDE	
	SSIM	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM	PSNR
$T = 1, K = 4$	0.866 ± 0.039	21.4 ± 1.23	0.859 ± 0.028	21.1 ± 1.31	$0.508^* \pm -$	$16.7^* \pm -$	0.858 ± 0.031	21.0 ± 1.20	0.823 ± 0.077	20.5 ± 1.41
$T = 1, K = 8$	0.861 ± 0.035	21.1 ± 1.10	0.859 ± 0.031	21.0 ± 1.25	$0.763^{**} \pm 0.103$	$19.3^{**} \pm 0.84$	0.865 ± 0.036	21.5 ± 1.93	0.817 ± 0.091	20.4 ± 1.47
$T = 1, K = 16$	0.762 ± 0.128	19.3 ± 1.32	0.826 ± 0.082	20.6 ± 1.55	0.769 ± 0.086	19.6 ± 1.02	0.855 ± 0.033	20.8 ± 1.35	0.736 ± 0.134	19.9 ± 1.66

Table 14. Prediction quality (SSIM, PSNR) for the pendulum dynamics tracking experiment, with respect to models trained using OT, Gumbel and Soft resampling. Results correspond to the first experiment setting.

Method	SSIM	PSNR
OT ($\varepsilon = 0.5$)	0.861 ± 0.033	21.1 ± 1.29
OT ($\varepsilon = 1.0$)	0.813 ± 0.109	20.6 ± 1.91
OT ($\varepsilon = 1.5$)	0.849 ± 0.031	20.8 ± 1.16
Gumbel (0.1)	0.846 ± 0.034	20.6 ± 1.22
Gumbel (0.3)	0.859 ± 0.029	21.1 ± 1.25
Soft (0.7)	0.860 ± 0.031	21.0 ± 1.27
Soft (0.9)	0.814 ± 0.105	20.5 ± 1.78

with resampling, and compare resamplers while holding the rest of the system fixed. Resampling itself can interact with the optimisation process in non-trivial ways, and we thus primarily evaluate the comparison by end-to-end performance. However, tempering the observation likelihood in the first setting reduces weight concentration: the weights become more uniform and ESS remains high. With this ESS-based adaptive resampling, the resampling can therefore be triggered infrequently. As such, to further stress test differentiable resampling in this context, we consider the second setting with more concentrated weights and resampling at nearly every filtering step.

As we have seen, even in this challenging setting, diffusion resampling remains competitive among the baselines. Figure 10 shows a qualitative comparison of a top performing model in each resampling class in this setting. Here, the diffusion resampler is among the three top-performing baselines. While soft resampling occasionally shows better performance in terms of final metrics compared to diffusion resampling, see Figure 9, we note that it also exhibits significantly worse weight degeneracy compared to diffusion resampling and the other baselines. In addition, it is by construction not fully differentiable (see Table 20). We also note that while OT produces the top performing model here, it exhibits high variability and is overall unstable in this setting, with only a few converging training runs.

Identifiability of the latent space Because both the transition model f_θ and the decoder r_ϕ are learnt only through the image observation likelihood, the latent state is typically identifiable only up to (possibly nonlinear) invertible transformations, and there is no unique correspondence between the learnt states and the true physical coordinates without special constructions (see, e.g., Greydanus et al., 2019, for a similar issue). For this reason, we do not directly evaluate the pendulum dynamics in state space, but instead assess model quality in the observation space via image reconstruction metrics. A similar evaluation strategy is common in recent work on latent SDEs for high-dimensional sequence data. For example, both Bartosh et al. (2025) and Course & Nair (2023) primarily report qualitative comparisons of generated image sequences in comparable settings, rather than quantitative metrics in the latent state space, reflecting the view that high-quality reconstruction is indicative of a meaningful latent representation. Nevertheless, we find that sometimes the learnt dynamics closely match the true pendulum dynamics up to a linear transformation of the latent coordinates. Figure 11 illustrates two such examples with the latent dynamics learnt using diffusion resampling, where a simple linear transformation of the latent trajectory yields a close match to the ground-truth pendulum state (coefficient of determination $R^2 \approx 0.98$ (left) and $R^2 \approx 0.92$ (right), respectively). This indicates that the models have learnt meaningful latent representations of the underlying pendulum dynamics, albeit in a transformed coordinate system.

Network architectures Both the neural network modelling the latent dynamics, f_θ , and the neural network modelling the decoder, r_ϕ , operate on a feature vector $\mathbf{h}(Z_j) = [\sin(Z_j^{(1)}) \quad \cos(Z_j^{(1)}) \quad Z_j^{(2)} / 10]$, which embeds the angle and scales

Table 15. Prediction quality in terms of structural similarity index (SSIM) and PSNR (higher the better) for the pendulum dynamics tracking experiment, with respect to models trained using diffusion resampling in the second experiment setting. Each result is presented in terms of mean over 9 independent runs, together with the corresponding standard deviation, recorded after 1500 training iterations. For four configurations only 8 out of 9 (*) runs, respectively, were non-divergent.

Method	Euler-Maruyama				Jentzen-Kloeden				Tweedie	
	SDE		ODE		SDE		ODE		SDE	
	SSIM	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM	PSNR
$T = 1, K = 4$	$0.671^* \pm 0.082$	$17.2^* \pm 0.666$	0.607 ± 0.091	17.2 ± 0.834	$0.517^* \pm 0.134$	$16.9^* \pm 0.528$	0.507 ± 0.048	16.5 ± 0.443	$0.549^* \pm 0.081$	$16.7^* \pm 0.666$
$T = 1, K = 8$	$0.576^* \pm 0.080$	$16.6^* \pm 0.542$	0.511 ± 0.070	16.7 ± 0.554	0.599 ± 0.101	17.1 ± 0.579	0.481 ± 0.074	16.5 ± 0.253	0.503 ± 0.067	16.5 ± 0.32
$T = 1, K = 16$	0.576 ± 0.082	16.6 ± 1.19	0.280 ± 0.112	16.1 ± 0.235	0.557 ± 0.101	16.9 ± 0.744	0.239 ± 0.082	16.0 ± 0.202	0.227 ± 0.074	16.1 ± 0.197

Table 16. Prediction quality (SSIM, PSNR) for the pendulum dynamics tracking experiment, with respect to models trained using OT, Gumbel and Soft resampling. Results correspond to the second experiment setting. For OT configurations, only 3 (**) and 4(*) runs were non-divergent.

Method	SSIM	PSNR
OT ($\varepsilon = 0.5$)	$0.662^* \pm 0.141$	$17.7^* \pm 1.782$
OT ($\varepsilon = 1.0$)	$0.575^{**} \pm 0.303$	$17.5^{**} \pm 1.924$
OT ($\varepsilon = 1.5$)	$0.708^{**} \pm 0.159$	$18.2^{**} \pm 2.73$
Gumbel (0.1)	0.509 ± 0.060	16.7 ± 0.400
Gumbel (0.3)	0.593 ± 0.083	16.8 ± 0.508
Soft (0.7)	0.720 ± 0.107	17.7 ± 0.707
Soft (0.9)	0.704 ± 0.114	17.3 ± 0.872

the velocity. While the system is initialised with physical coordinates Z_0 , both networks are unconstrained and unregularised for all $t > 0$. As discussed above, this means that the system is free to learn a latent manifold sufficient for image reconstruction, even if the latent representation is a non-linear transformation of the true physical coordinate system. We employ a SIREN (Sitzmann et al., 2020) architecture to model the latent dynamics. The network takes the concatenated input $[\mathbf{h}(Z_j) \quad \zeta_j] \in \mathbb{R}^5$ and outputs the state derivative. The update rule in the first setting is $Z_{j+1} = Z_j + \text{Net}([\mathbf{h}(Z_j) \quad \zeta_j]) \Delta_j$. In the second setting, where we invoke resampling at nearly all filtering steps, we make an additional additive noise assumption, omit concatenating the input and let $Z_{j+1} = Z_j + \text{Net}(\mathbf{h}(Z_j)) \Delta_j + \zeta_j$. Without this assumption, the neural network is free to learn to neglect the noise, and in the first setting, we indeed found that the learnt dynamics network often had this issue. This can happen in end-to-end learning since minimising the optimisation objective does not necessarily require the network to optimise for a good filtering distribution, and reducing the randomness might, for instance, help stabilise optimisation.

In both settings, the network consists of 3 hidden layers of 256 units with sine activations ($\omega_0 = 8.0$ in the first layer). The decoder maps $\mathbf{h}(Z_j)$ to observations and is identical in both settings. It first processes the input via two linear layers (16 and 256 units), reshaping the output into a spatial feature map of size $4 \times 4 \times 16$. This is followed by three transposed convolution layers (kernel size 3×3 , stride 2) with output channels of 32, 16, and 1, respectively, to upsample to the final 32×32 image. ReLU activations are used for all hidden layers, while the output layer is linear.

L. Bayesian neural network training

We apply SMC for a larger-scale problem: training a (partial) Bayesian neural network (Zhao et al., 2024; Sharma et al., 2023, pBNN). There are two steps moving from training a classical BNN to training a pBNN with SMC sampler. First, pBNNs only have priors on a (small) subset of the neural network parameters. This results in a latent-variable model to learn from the data, where computing the posterior distribution is easier than that of the full BNN. Let us denote the pBNN by $x \mapsto f_{\theta,z}(x)$, where θ and z stand for deterministic and stochastic parameters, respectively. Second, the prior construction is no longer static or explicit (e.g., z being a Gaussian). Instead, z is assumed to follow a Markovian dynamics motivated by Zhao et al. (2024); Chang et al. (2022); Freitas et al. (2000) so that data are modelled as independent conditionally on a latent process. Specifically, we apply the pBNN setting and construct an SSM

$$\begin{aligned} p(z_j | z_{j-1}) &= \mathcal{N}(z_j; \rho z_{j-1}, 1 - \rho^2), \\ p_{\theta}(D_j | z_j) &= \text{Softmax}(f_{\theta,z_j}; D_j), \end{aligned} \tag{38}$$

for CIFAR10 classification, where D_j is a batch data at surrogate time j (training step), and we choose $\rho = 0.99$.

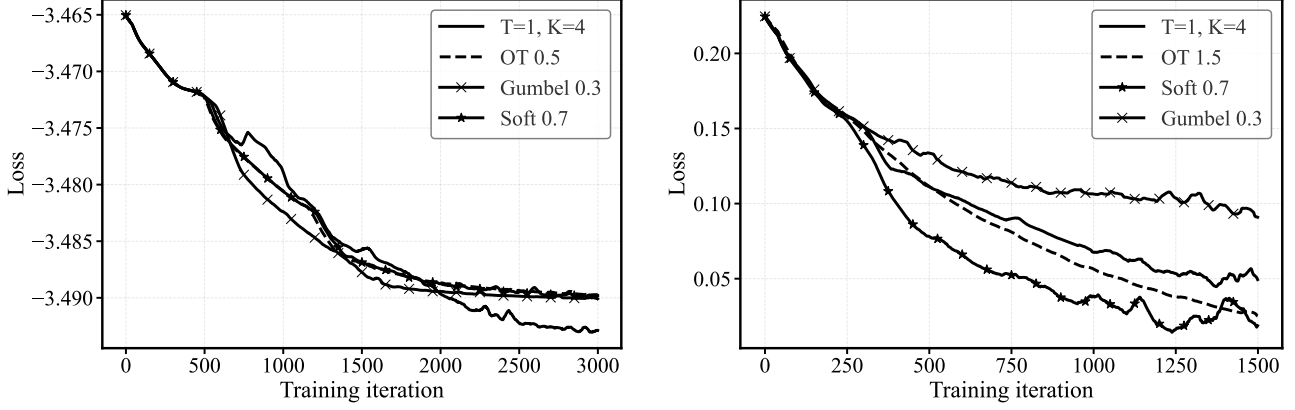


Figure 8. (Left) Median loss during the training process for the top performing configuration in each resampling class in the first setting. We note that diffusion resampling achieves the lowest loss. (Right) Median loss during the training process for the top performing configuration in each resampling class in the more challenging setting. We note that for OT, the reported median loss is based on the few successful runs and is therefore not statistically meaningful. While Soft achieves the lowest median loss (see the text for discussion), diffusion resampling remains competitive and yields low, stable loss, whereas Gumbel performs markedly worse than all baselines.

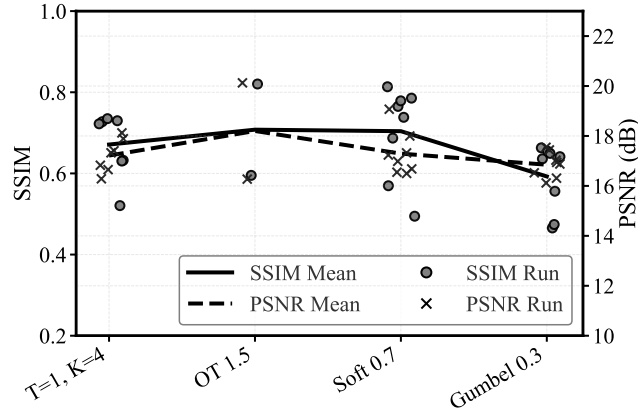


Figure 9. Mean prediction SSIM and PSNR for the best (by mean) model configuration of each resampler in the more challenging setting. Individual runs are shown as scatter points. We find that although Soft can occasionally give better results than diffusion resampling, its results exhibit large variability. We also observe that Gumbel performs worse than diffusion resampling. We further observe that OT gets top SSIM and PSNR here, but note that it exhibits large variability and is not stable in this task (most OT runs diverged during training).

Table 17. CIFAR10 classification with different resampling methods. Results are averaged over 5 independent trainings.

	Diffusion	OT ($\varepsilon = 1$)	Gumbel (0.3)	Soft (0.7)	Stopped
Accuracy	87.81 $\pm$ 0.64	83.28 $\pm$ 0.63	86.92 $\pm$ 0.58	86.57 $\pm$ 0.58	86.28 $\pm$ 0.57
F1 score	87.76 $\pm$ 0.63	83.18 $\pm$ 0.70	86.89 $\pm$ 0.57	86.54 $\pm$ 0.57	86.21 $\pm$ 0.56

We choose a ResNet18 neural network, and set the first two convolution layers be stochastic (i.e., the neural network parameters in this part is the latent variable z). This results in latent dimension $d_z = 1,856$ and parameter dimension $d_\theta = 11,172,106$. We choose the number of particles to be 8, resampling threshold 0.5, and apply an Adam optimiser train for 200 epochs. The diffusion resampling parameter is $T = 1$ and $K = 4$ using the Jentzen–Kloeden SDE integrator. The results of the classification evaluated using accuracy and F1 score is shown in Table 17.

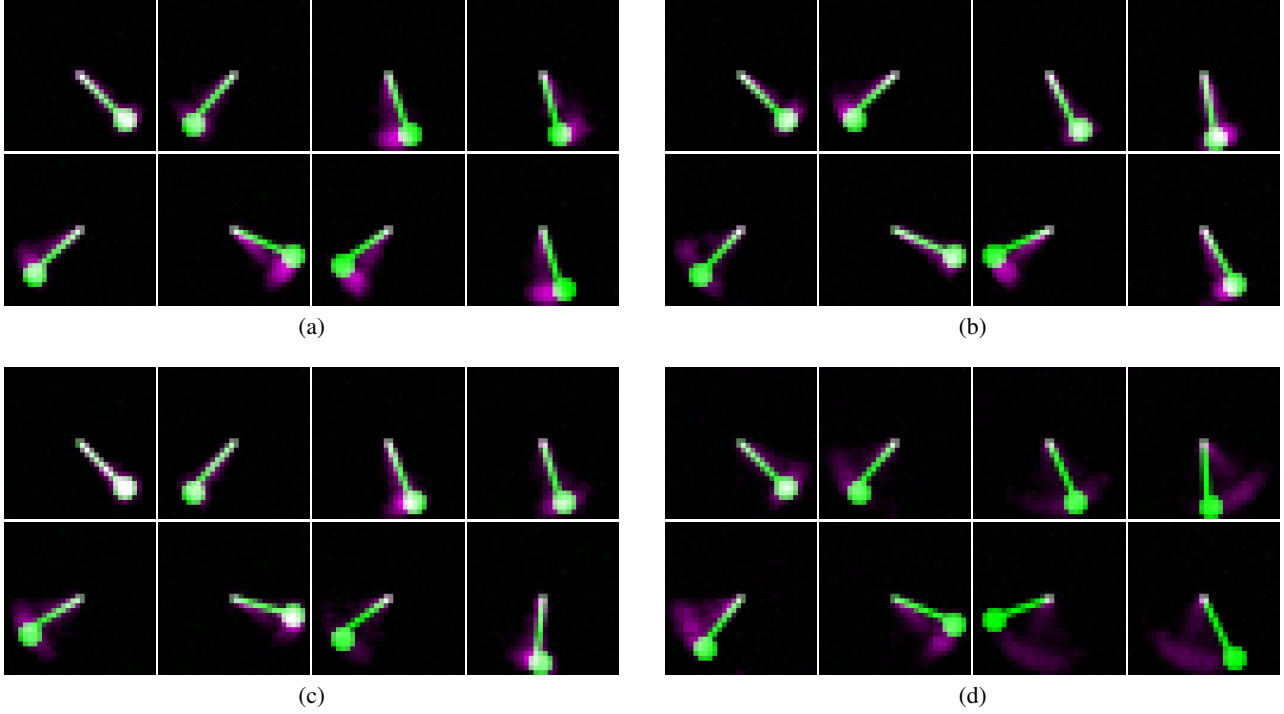


Figure 10. Qualitative comparisons of the learnt pendulum dynamics trained end-to-end using a differentiable resampler in the SMC training loop. The results correspond to the second, more challenging experiment setting using resampling at nearly all filtering steps. The ground truth (green) is overlaid with model predictions (purple). White pixels indicate perfect alignment, while coloured regions highlight positional discrepancies (e.g., phase lag). Snapshots are shown at eight evenly spaced time points over the 4 second simulation (read from left to right and top to bottom). Each panel shows a top result in terms of mean SSIM for each differentiable resampling method in our comparison: a) Diffusion (mean SSIM/PSNR 0.761/17.0, b) Soft (0.816/19.2), c) OT (0.820/20.1), d) Gumbel (0.663/16.9).

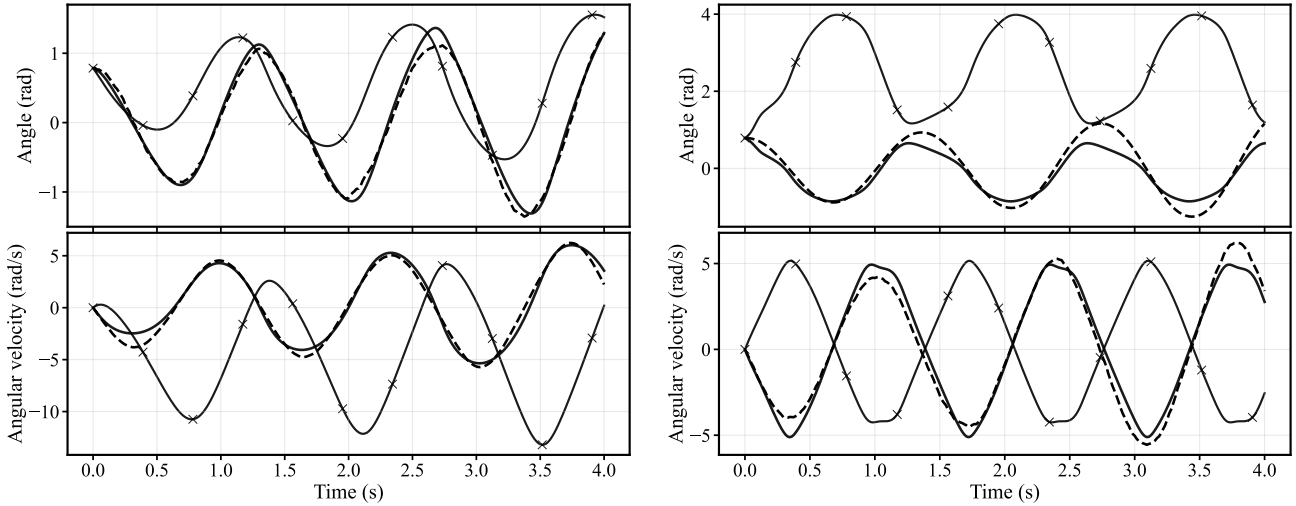


Figure 11. (Left) Ground-truth pendulum state trajectories (dashed) and predictions in the original neural (solid with crosses) and transformed (solid) coordinate systems, for a top-performing model trained with diffusion resampling ($T = 1$, $K = 4$) in the first setting. The dynamics model is trained until convergence, and correspond to the image predictions presented in Figure 6. (Right) Same as the previous image, but for a model optimised in the more challenging setting. The corresponding image predictions are presented in Figure 10a.

Table 18. Prediction quality in terms of MSE ($\times 10^{-3}$, lower is better) for the weather forecasting experiment in the full year setting. Results are presented as mean $\pm$ standard deviation over 5 independent runs recorded after 3000 training iterations. Train MSE is averaged over 5 evenly spaced windows from each of the 7 training years; eval MSE is computed on 5 evenly spaced windows from the held-out year.

Method	Train MSE (masked)	Eval MSE (full)
Diffusion, Euler–Maruyama ($T = 1, K = 4, \text{ODE}$)	2.2 ± 0.7	2.1 ± 0.8
Diffusion, Euler–Maruyama ($T = 1, K = 4, \text{SDE}$)	2.5 ± 0.8	2.4 ± 0.7
Diffusion, Euler–Maruyama ($T = 1, K = 8, \text{ODE}$)	2.1 ± 0.5	2.0 ± 0.4
Diffusion, Jentzen–Kloeden ($T = 1, K = 8, \text{ODE}$)	2.0 ± 0.5	1.9 ± 0.4
Soft (0.7)	1.7 ± 0.1	1.7 ± 0.1
Soft (0.9)	1.6 ± 0.1	1.7 ± 0.2
Gumbel (0.1)	4.6 ± 2.0	4.7 ± 2.3
Gumbel (0.3)	1.7 ± 0.1	1.7 ± 0.1
OT ($\varepsilon = 0.5$)	1.9 ± 0.1	1.8 ± 0.1
OT ($\varepsilon = 1.0$)	1.8 ± 0.2	1.8 ± 0.1
OT ($\varepsilon = 1.5$)	2.4 ± 1.2	2.4 ± 1.4

M. Weather forecast

Experiment settings As another large-scale experiment, we consider a high-dimensional weather forecasting task to further demonstrate that the diffusion resampler scales to high-dimensional state space models. Given partial noisy observations, we track the evolution of the 850 hPa atmospheric temperature field over time. We use SMC to enable learning the parameters θ of an SSM with atmosphere dynamics model $p_\theta(z_k | z_{k-1})$ and observation model $p(y_k | z_k)$. Here, the latent state $z_k \in \mathbb{R}^{2048}$ represents the global temperature field at 5.625° resolution, with the resulting latent dimension $d = 2048$ (32×64 spatial grid). The partial observation y_k consists of 80% of pixels selected uniformly at random at each time step, corrupted by Gaussian noise with standard deviation $\sigma_y = 0.01$. The observation model is $p(y_k | z_k) = \mathcal{N}(y_k | \mathcal{M}_k z_k, \sigma_y^2 I)$, where $\mathcal{M}_k$ denotes the random mask operator for the observed pixels at time step k . A UNet parametrises the transition density $p_\theta(z_k | z_{k-1})$, and is learnt using the negative log-likelihood estimate provided by a particle filter with resampling at every step.

The models are trained using ERA5 data from the WeatherBench benchmark dataset (Rasp et al., 2020). We present results for two different settings: a full year setting, where the model is tasked with iterative 4-day forecasts over a full year, and a single-sequence setting, where the model is tasked with iterative 8-day forecasts in January.

Training details We use data from the years 2010–2018 for training (excluding 2016), with 2014 held out for evaluation. The temperature fields are normalised to $[0, 1]$ using global min-max normalisation computed across all years. The UNet takes the current state z_{k-1} and process noise concatenated along the channel dimension. The output is given by $z_k = \text{Sigmoid}(z_{k-1} + \delta_\theta(z_{k-1}, q_k))$, where δ_θ is the UNet output, and $q_k \sim \mathcal{N}(0, \sigma_q^2 I)$ with $\sigma_q^2 = 0.1$. We train the model for 3000 iterations using the Adam optimiser with learning rate 2×10^{-4} and gradient clipping with global norm threshold 1.0. The optimiser omits at most 10 consecutive non-finite parameter updates by leaving the current parameter values unchanged. We use $N = 16$ particles and trigger resampling at each filtering step. By default, the diffusion resampler jitters the diagonal of the Gaussian reference by 10^{-5} for numerical stability, and the log-likelihood at each time step is normalised by the number of observed pixels.

Results and evaluation The learnt dynamics models are evaluated on unobserved (masked) pixels from the training sequences and on full sequences from the held-out year. We use the mean square error (MSE) averaged over time steps and 16 individual rollout samples to evaluate the performance. All MSE values are reported at scale $\times 10^{-3}$, averaged over 5 independent training runs. Table 18 shows results for the full year setting, where training sequences of length 16 are randomly sampled from the full year of 6-hourly data across all 7 training years. The model is evaluated on 5 evenly spaced windows from the held-out year, covering all seasons. Table 19 shows results for the single-sequence setting, where the training data consists of fixed sequences of length 32 covering the first 8 days of January for each year in the training data. The model is evaluated on the corresponding January sequence from the held-out year.

Our results show that diffusion resampling can be used inside the SMC learning pipeline to learn atmosphere dynamics in this complex, high-dimensional setting. Tables 18 and Table 19 show that almost all configurations achieve relatively low MSE on both unobserved pixels in sequences drawn from the training data, and on unseen sequences drawn from the

Table 19. Prediction quality in terms of MSE ($\times 10^{-3}$, lower is better) for the weather forecasting experiment in the single-sequence setting. Results are presented as mean $\pm$ standard deviation over 5 independent runs recorded after 3000 training iterations. Train MSE is averaged over all 7 training years; eval MSE is computed on the January sequence from the held-out year.

Method	Train MSE (masked)	Eval MSE (full)
Diffusion, Euler–Maruyama ($T = 1, K = 4, \text{ODE}$)	0.6 ± 0.1	1.9 ± 0.1
Diffusion, Euler–Maruyama ($T = 1, K = 4, \text{SDE}$)	0.6 ± 0.1	2.0 ± 0.1
Diffusion, Euler–Maruyama ($T = 1, K = 8, \text{ODE}$)	0.5 ± 0.0	2.0 ± 0.0
Diffusion, Jentzen–Kloeden ($T = 1, K = 8, \text{ODE}$)	0.5 ± 0.0	2.0 ± 0.1
Soft (0.7)	0.5 ± 0.1	2.1 ± 0.1
Soft (0.9)	0.5 ± 0.1	2.0 ± 0.1
Gumbel (0.1)	0.6 ± 0.1	2.0 ± 0.0
Gumbel (0.3)	0.6 ± 0.1	1.9 ± 0.0
OT ($\varepsilon = 0.5$)	0.6 ± 0.1	1.9 ± 0.1
OT ($\varepsilon = 1.0$)	0.6 ± 0.0	1.9 ± 0.1
OT ($\varepsilon = 1.5$)	0.6 ± 0.1	2.0 ± 0.1

held-out evaluation data. The worst results are obtained by Gumbel ($\tau = 0.1$) in the full year setting. In this setting, which contains data from the annual temperature variations, all resamplers struggle to learn fine-grained details in the current setup. Soft resampling achieves the lowest eval MSE with low variance, while the best diffusion configuration (Jentzen–Kloeden, $K = 8$, ODE) achieves competitive performance. We note that these results rely on only 5 independent training runs for each model configuration. The standard deviation is not negligible in the comparison, and some configurations in the full year setting (including some diffusion configurations) exhibit relatively high variance.

Figure 12 shows examples of predicted forecasts using a model trained with diffusion resampling. These results correspond to the second experiment setting, where models are trained on a small dataset from a limited season (early January). Though Table 19 may indicate some overfitting, the evaluation forecasts show that some generalisable dynamics have been learned, and that the forecasts include fine-scale features.

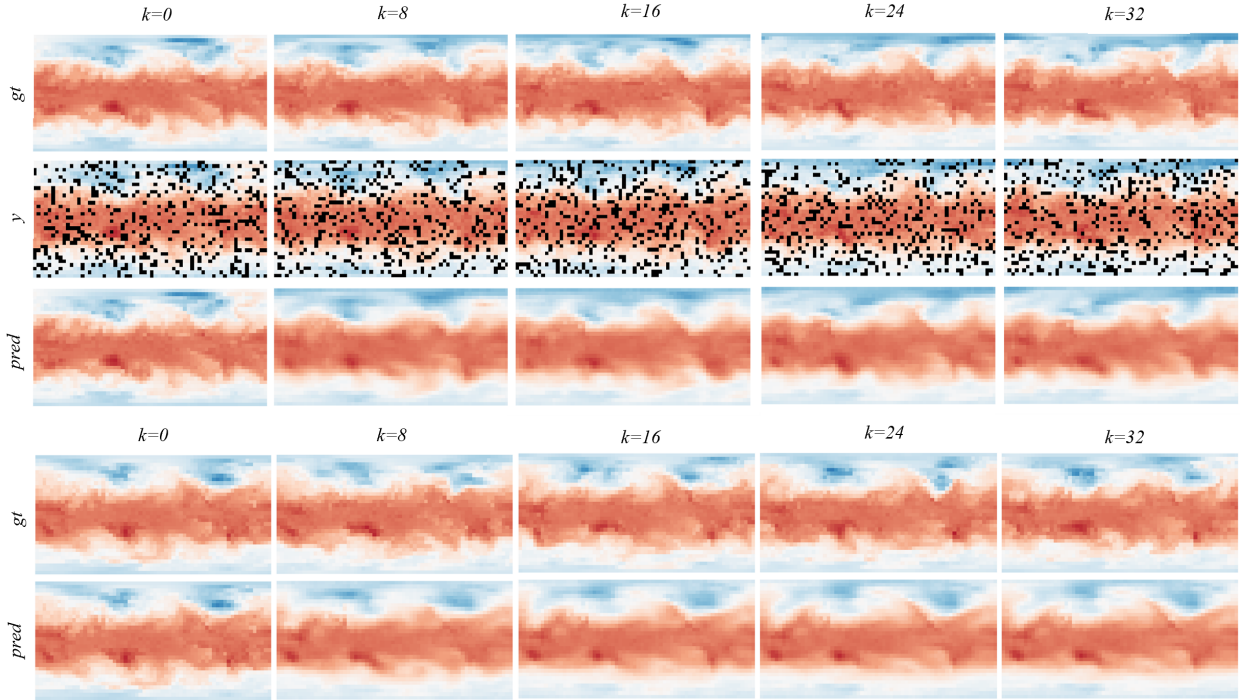


Figure 12. Top panel: Training sequence forecast with corresponding masked observations (middle row). Bottom panel: Evaluation sequence forecast from the held-out year. Each panel shows ground truth (top row) and model prediction (bottom row) at evenly spaced time steps. The model was trained using the diffusion resampler in the learning pipeline.

For a more comprehensive comparison, there are limitations to address, such as the limited number of independent runs, choice of evaluation metric and the amount of data used for training. A simple MSE against the ground truth does not fully capture the predictive distribution quality and may not fully capture the forecasting performance in terms of fine-grained details and long forecasting windows. In both settings, the diffusion resampler achieves a performance comparable to the baselines. Training is stable across seeds, which shows that the diffusion resampler scales at latent dimension $d = 2048$.

N. Choosing the hyperparameters

Although the diffusion resampling is powerful, it comes with a variety of hyperparameters to tune: the diffusion coefficients b and π_{ref} , the diffusion time T , the integrator, and integration step K . Ideally, π_{ref} should well approximate the underlying continuous distribution, and the choice b should correspondingly reflect the approximation error as in Corollary 1. The setting of diffusion time T is arbitrary if one can sample p_T exactly, but smaller T leads to potentially lower integration steps required. As for the integrator and integration steps, our empirical results indicate that Jentzen–Kloeden and Euler–Maruyama are often a good start, and $K \leq 8$ is often sufficient for learning large neural network-parametrised systems. To retain a good computational complexity, it is suggested that the number of integration steps should be smaller than the number of samples: $K < N$.

O. Additional related work

In addition to the discussion in Section 5, there are also connections to other, less central, lines of work. For instance, the ensemble score estimator is related to the kernel projection used in Stein variational gradient flow (Liu, 2017). The controlled gain term in Yang et al. (2013) also plays a role analogous to resampling in a continuous-time limit, but involves a computationally demanding PDE-solving step. While there are interesting connections to our work, the approach focuses on continuous-time flow filtering (cf. Kang et al., 2025) and leads to algorithms that are no longer standard SMC. Bao et al. (2024) propose an ensemble filter where the update step was replaced by a conditional version of the diffusion in Equation (13). This too targets at Equation (30) but additionally contains errors from likelihood score approximation.

P. Take-away messages

We here provide a TL;DR summary of the main findings, with Table 20 giving an overall comparison among commonly used differentiable resampling schemes.

- Diffusion resampling largely excels for parameter estimation in state-space models (SSMs) and is useful for practical applications. It is computationally fast, and provides consistent and stable gradient estimates.
- Diffusion resampling is useful not only for differentiation, but also for reducing resampling error in general. The primary reason for this is that diffusion resampling can take additional information (e.g., π_{ref}) of the target into account. When we have a complete information $\pi_{\text{ref}} = \pi$, the diffusion resampling becomes an optimal resampling algorithm. Corollary 1 explicitly shows how the resampling error can be controlled depending on how well π_{ref} approximates π , calibrated by b . In contrast, multinomial resampling only uses information from the given samples $\{(w_i, X_i)\}_{i=1}^N$ which contains incomplete and less information compared to diffusion resampling using the continuous distribution approximation.
- For pure filtering tasks, even without focusing on parameter estimation, the diffusion resampling generally outperforms the peer methods.
- If a good reference distribution is hard to construct, diffusion resampling may not be optimal for a pure resampling problem outside the SSM context. For instance, as shown in the Gaussian mixture experiment, the large discrepancy between the Gaussian reference and the highly non-Gaussian target may prolong the diffusion time. However, for SSMs, thanks to their sequential structure, constructing an effective reference becomes straightforward.

Table 20. Comparison among commonly used resampling schemes with their calibration parameters. The computational complexity is analysed per sample which can be embarrassingly parallelised over the samples. The complexity of OT is given by Luo et al. (2023a), where we here parametrise the Sinkhorn precision with the regularisation parameter ε to unify comparison. By “fully differentiable” we mean that the pathwise gradient is well defined, for instance, the soft resampling is only partially differentiable.

Method	Fully differentiable	Consistent	Unbiased	Computational complexity per re-sample
Diffusion K	Yes	Yes, as $N(K) \rightarrow \infty$	No	$O(K N)$
OT ε	Yes	Yes, as $N(\varepsilon) \rightarrow \infty$	No	$O(N^2 \log(N)^{-1} \varepsilon^{-1})$
Gumbel τ	Yes, but not $\tau \rightarrow 0$	No, except at $\tau \rightarrow 0$	No	$O(N)$
Soft α	No	Yes, as $N \rightarrow \infty$	No	$O(N)$
Multinomial	No	Yes	Yes, as $N \rightarrow \infty$	$O(N)$